

程序设计实习（实验班-2026春）

距离度量及其计算：低维欧氏距离

授课教师：姜少峰

助教：方心童、葛致远、李雷思问、刘鹏飞

Email: shaofeng.jiang@pku.edu.cn

欧氏距离

- 是最常见的向量的距离度量

- $\forall x, y \in \mathbb{R}^d, \|x - y\|_2 = \sqrt{\sum_{i=1}^d (x_i - y_i)^2}$

- $\|\cdot\|_2$ 是 ℓ_2 -norm (2-范数)

- 欧氏空间有三角形不等式: $\forall x, y, z \in \mathbb{R}^d, \|x - y\| + \|y - z\| \geq \|x - z\|$

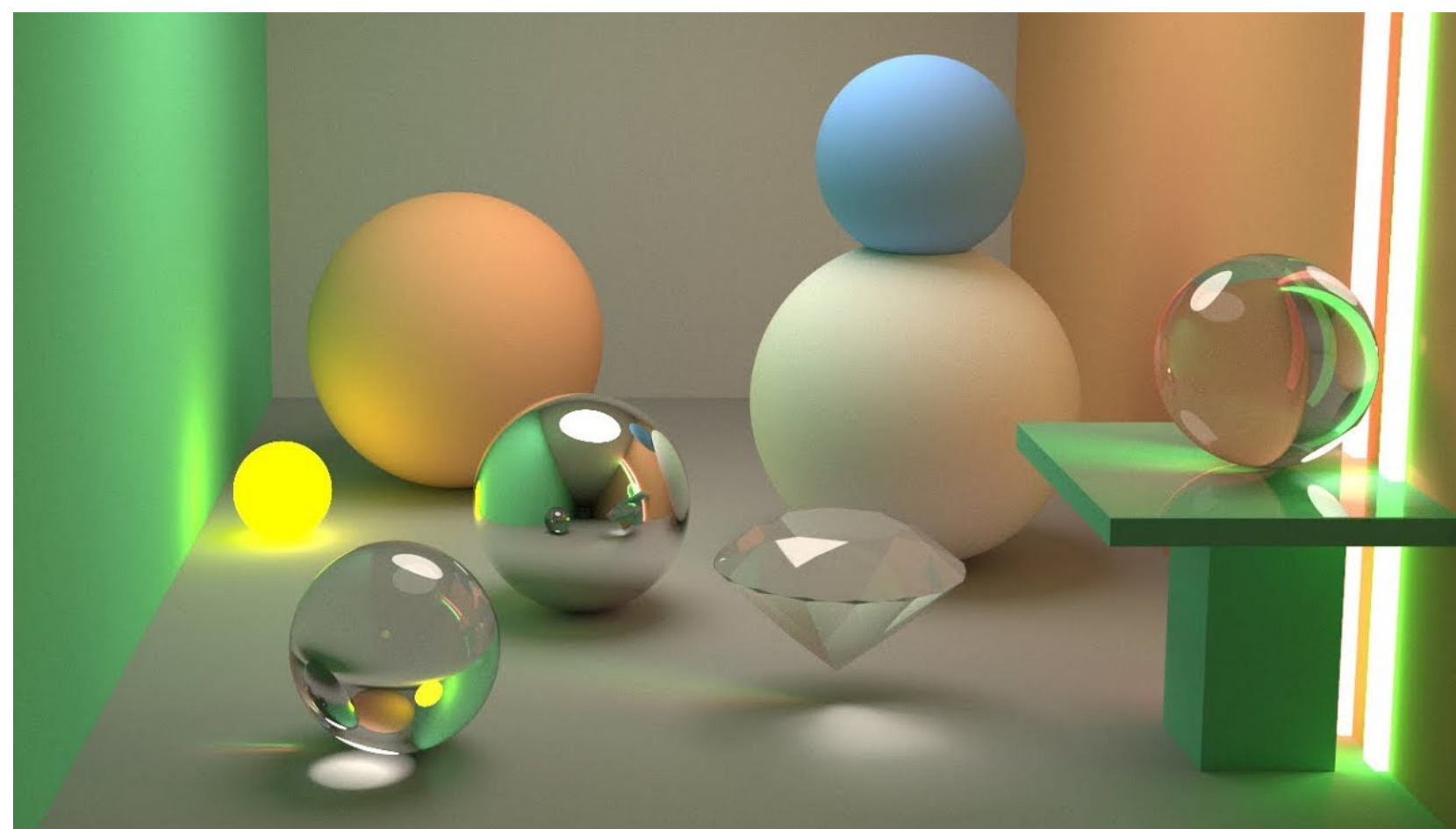
为何重要?

- 我们生存在3维欧氏空间里，距离自然定义为欧氏距离
- computer graphics, computer vision等也主要在欧氏空间内研究

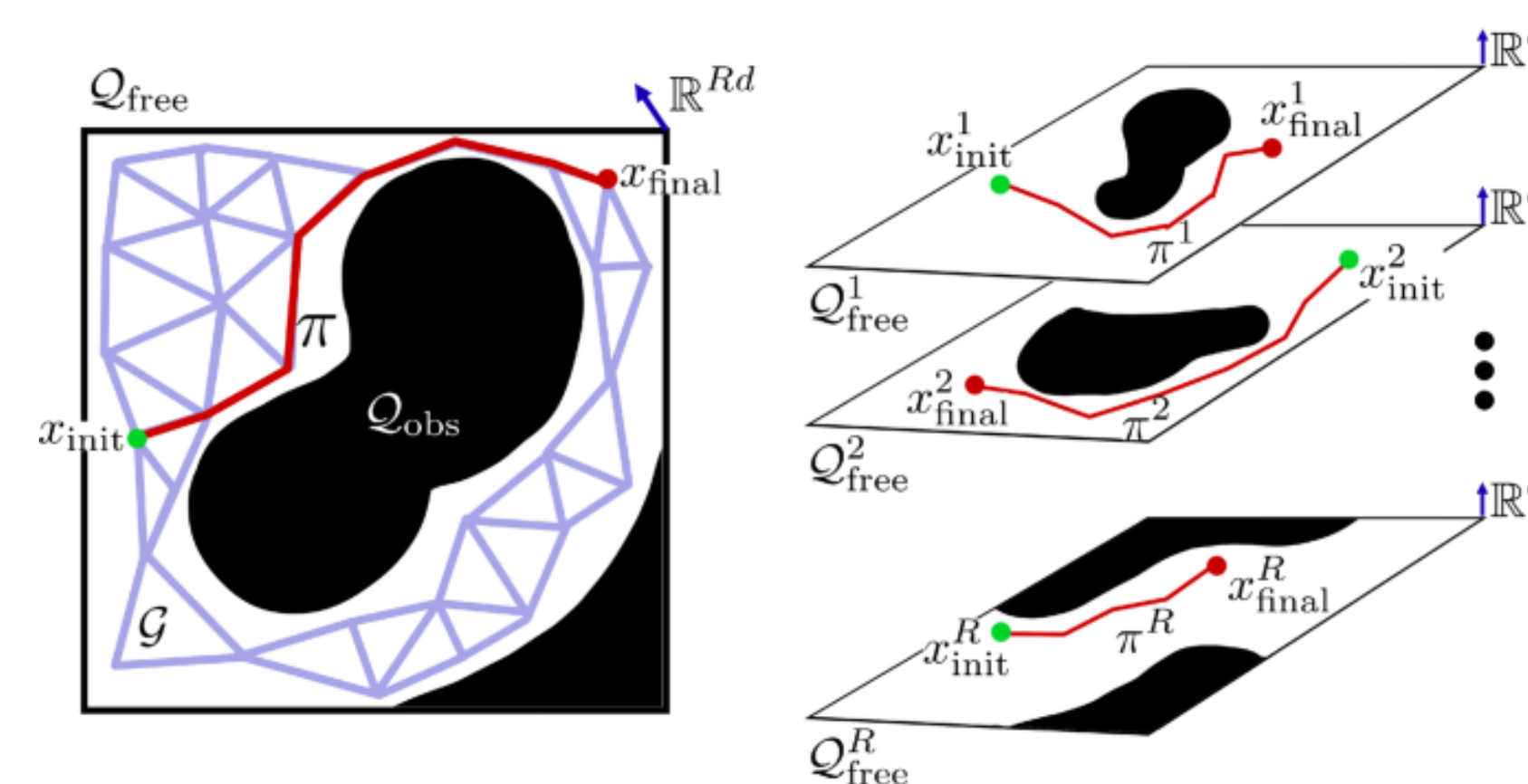
VR



渲染

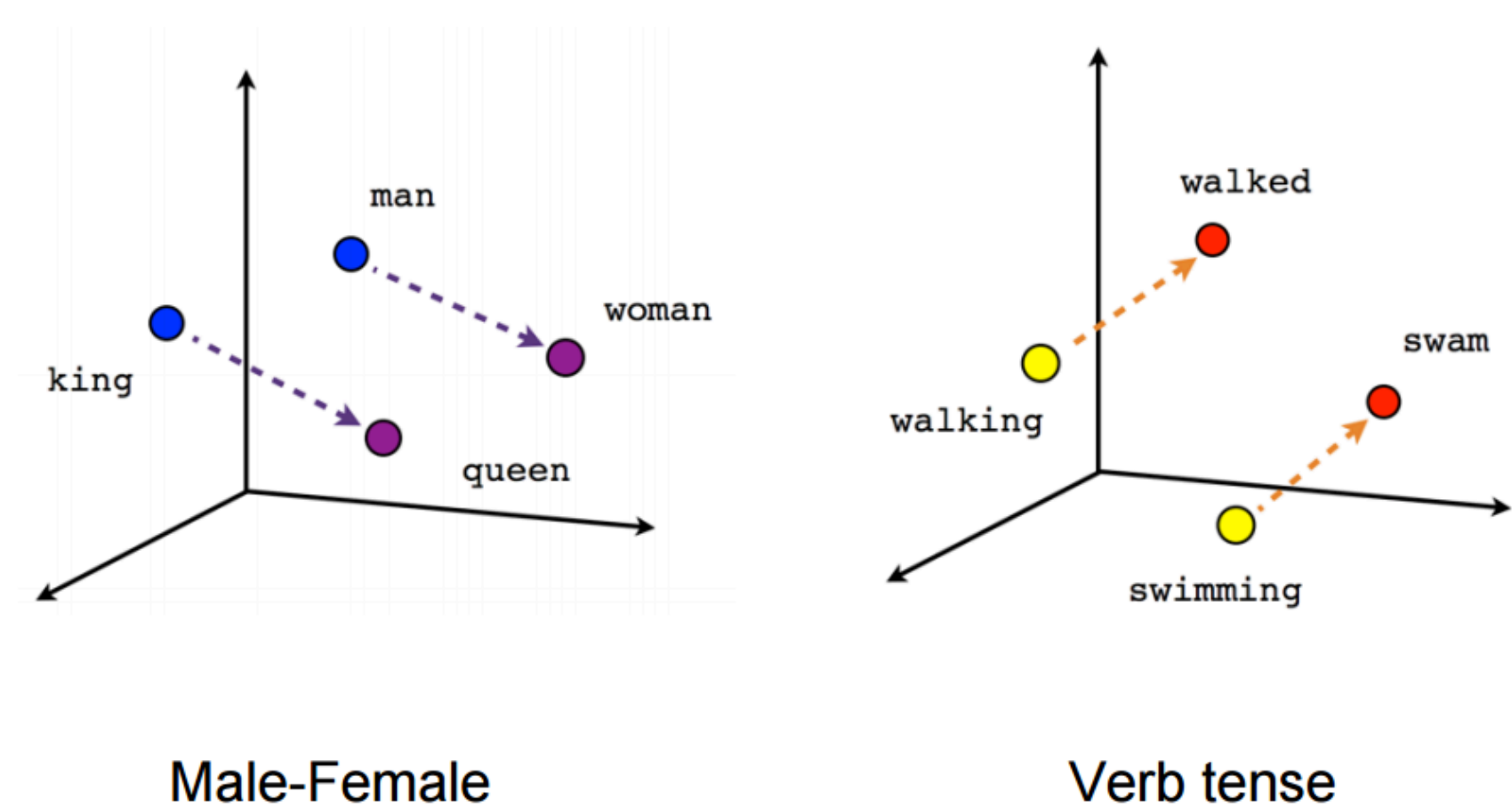


机器人运动规划

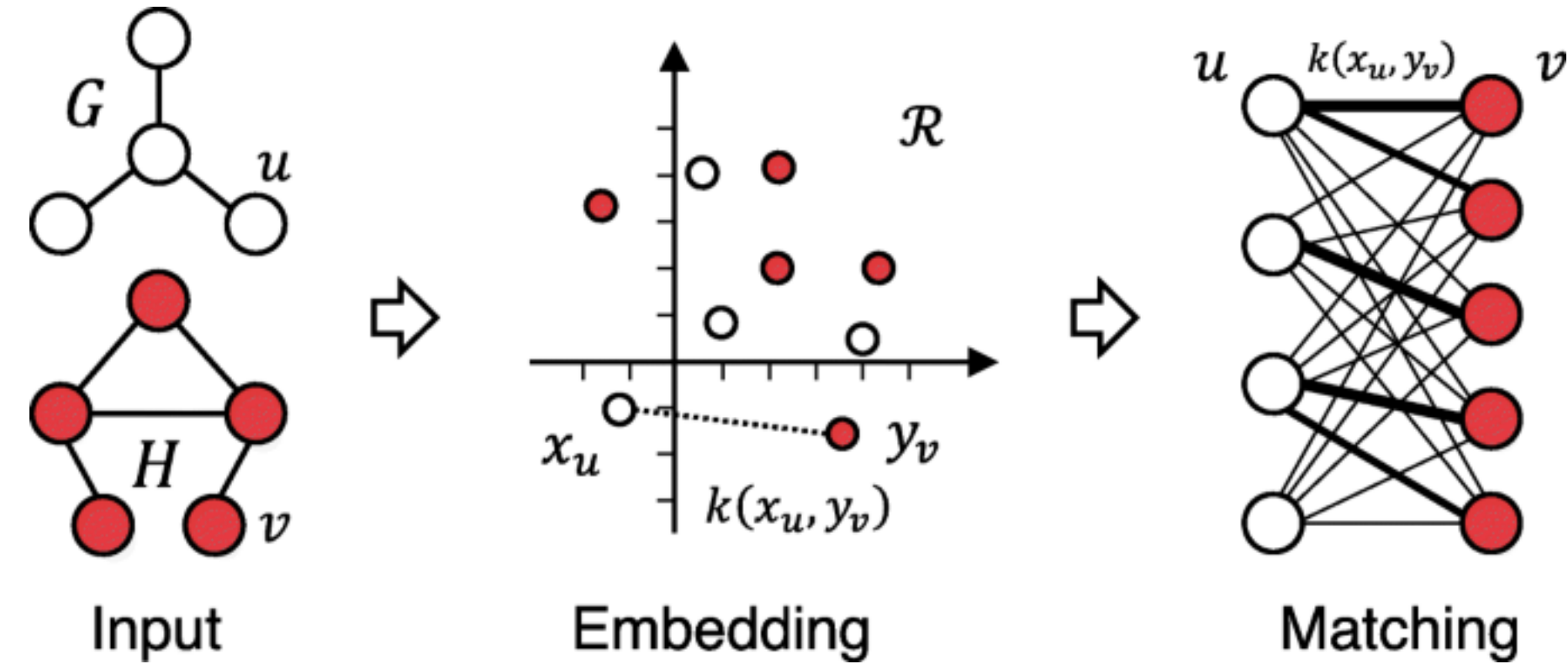


其他数据类型也经常映射到欧氏空间来处理

Word Embedding



Graph Embedding



Kernel与欧氏空间

- Kernel是一种一般的将数据点映射到高维欧氏空间的方法
- 设数据集是 U ，那么kernel function的形式是 $K : U \times U \rightarrow \mathbb{R}_+$
- 一个函数 K 是“合法”kernel，如果存在 $\varphi : U \rightarrow \mathbb{R}^t$ 使得 $\forall x, y \in U$,

$$K(x, y) = \langle \varphi(x), \varphi(y) \rangle$$

φ 将数据点映射成（高维）向量
 $K(x, y)$ 就是对应的内积，是一种相似度度量
可以类比cosine similarity

- 例如Gaussian kernel: $K(x, y) = \exp(-\|x - y\|_2^2)$
- 一般只给出 K ， φ 并不显式给出，在不直接知道 φ 的情况下进行欧氏空间上的计算

关于维度 d

Curse of Dimensionality

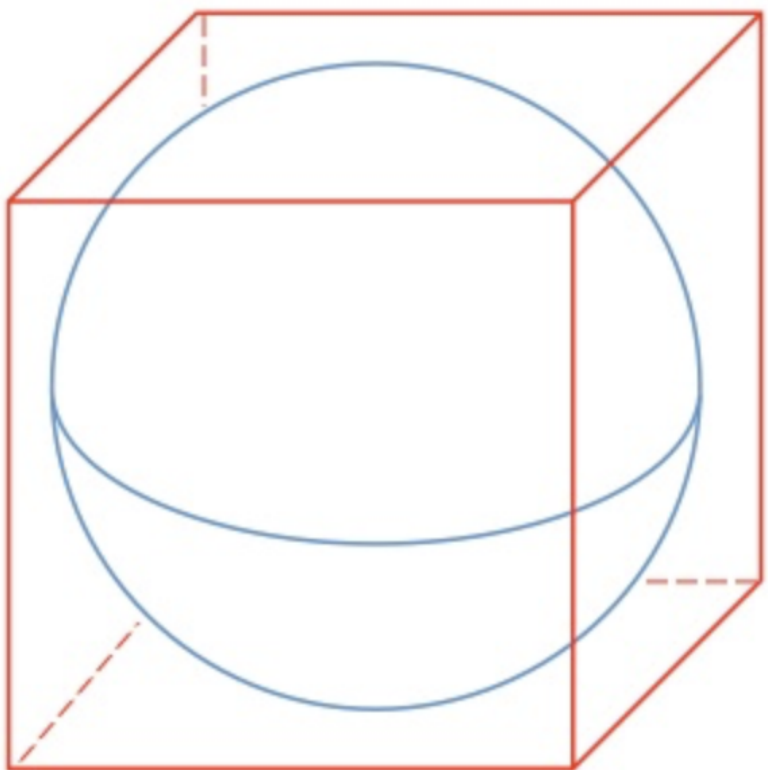
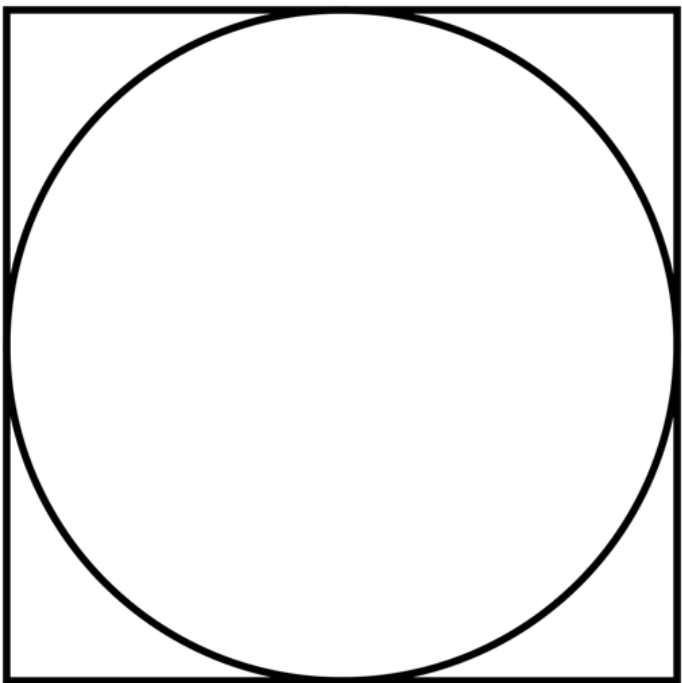
高维欧氏空间的“奇怪”特性

区别于低维空间上的常见直觉

- 高维空间还有很多其他“奇怪”的特性
- 方 vs 圆：单位立方体的体积是1，单位球的体积是 $O(2^{-O(d)})$

d维半径为r的球体积近似：
$$V_d(r) \approx \frac{1}{\sqrt{d\pi}} \left(\frac{2\pi e}{d} \right)^{d/2} \cdot r^d$$

- 另：单位圆和方的直径差 $\sqrt{d}$ 倍

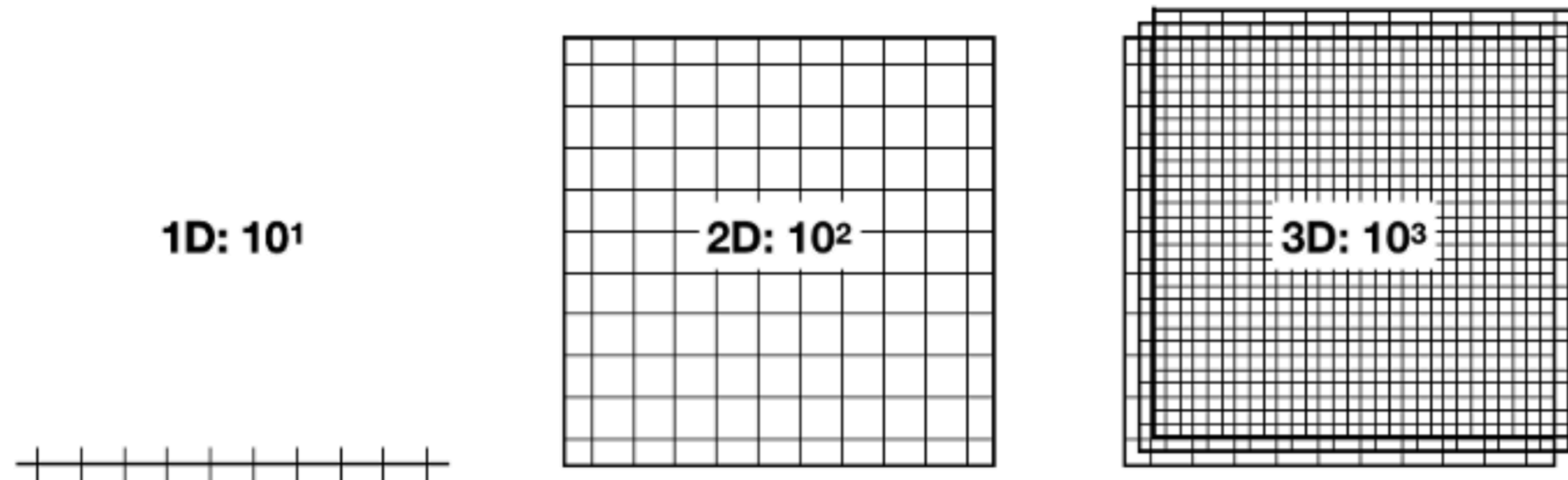


Dimension	Volume of a ball of radius R
0	1
1	$2R$
2	$\pi R^2 \approx 3.142 \times R^2$
3	$\frac{4\pi}{3} R^3 \approx 4.189 \times R^3$
4	$\frac{\pi^2}{2} R^4 \approx 4.935 \times R^4$
5	$\frac{8\pi^2}{15} R^5 \approx 5.264 \times R^5$
6	$\frac{\pi^3}{6} R^6 \approx 5.168 \times R^6$
7	$\frac{16\pi^3}{105} R^7 \approx 4.725 \times R^7$
8	$\frac{\pi^4}{24} R^8 \approx 4.059 \times R^8$
9	$\frac{32\pi^4}{945} R^9 \approx 3.299 \times R^9$
10	$\frac{\pi^5}{120} R^{10} \approx 2.550 \times R^{10}$
11	$\frac{64\pi^5}{10395} R^{11} \approx 1.884 \times R^{11}$
12	$\frac{\pi^6}{720} R^{12} \approx 1.335 \times R^{12}$
13	$\frac{128\pi^6}{135135} R^{13} \approx 0.911 \times R^{13}$
14	$\frac{\pi^7}{5040} R^{14} \approx 0.599 \times R^{14}$
15	$\frac{256\pi^7}{2027025} R^{15} \approx 0.381 \times R^{15}$
n	$V_n(R)$

Curse of Dimensionality

样本稀疏

- 考虑呈每一维坐标的均匀分布以及 n 个样本点 P
 - 有 2^d 个潜在样本，因此经常 $n \ll 2^d$
 - 这导致数据极为“稀疏”，很难形成对整个样本空间的有效覆盖/估计



Curse of Dimensionality

对于计算问题的挑战

- 关于 d 的依赖经常是指数级的!

需要在某些例如 $P \neq NP$ 的复杂度假设下

- 在若干计算模型和问题中, $d = O(\log n)$ 已经很难!

[Trevisan, SICOMP 00]

- 例如, 对于欧氏TSP, 无多项式时间 $(1 + \epsilon)$ -近似

- 对于最近点对和直径问题, 无 $n^{2-\delta}$ 时间 $(1 + \epsilon)$ -近似算法

[Karthic et al., ITCS 19]; [Chen, CCC 18]

- 对于数据流MST/FL, 无 $o(n)$ 空间 $(1 + \epsilon)$ 近似

[Chen et al., STOC 23]; [Czumaj et al., FOCS 22]

$d = \Theta(\log n)$ 已经是高维
很多问题高维下无 $(1 + \epsilon)$ 近似

2^d vs $\text{poly}(d)$

- 对于算法来说，关于 d 的依赖常常满足一个二分性：
 - 要么是 $\text{poly}(d)$ ，要么是至少 2^d
 - 尤其是 $2^{\log^2 d}$ 这种quasi-poly的很罕见
- 因此： $\text{poly}(d)$ 的是适用于高维的算法，而任意的 $f(d)$ 的则是适用于低维的

例如 2^{2^d} 也是可以接受的

- 另一种理解方式是：低维即 $d = O(1)$ ，高维即 $d = O(\log n)$

低维欧氏距离上的高效近似技术

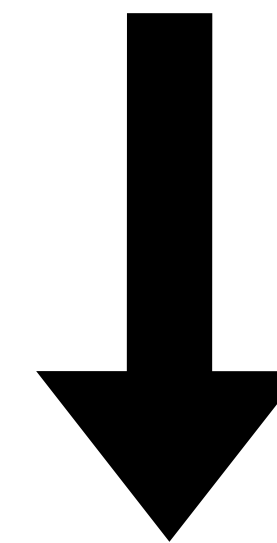
问题设定

我们主要以 $d = 2$ 进行讨论，但所有算法很容易推广到一般 d 维

- 输入 n 个 d 维、坐标在 $[\Delta] = \{1, \dots, \Delta\}$ 范围的数据点，即数据都在 $[\Delta]^d$ 内

例如考虑unsigned int, $\Delta = 2^{32}$

- 格点离散化技术：直径、最小包围球
- 四分树技术：最近邻查询
- WSPD = Well Separated Pair Decomposition：全局最近点对，最小生成树



层层递进，后者的构造基于前者

核心思想：离散化/找代表点

1. 格点离散化

格点离散化

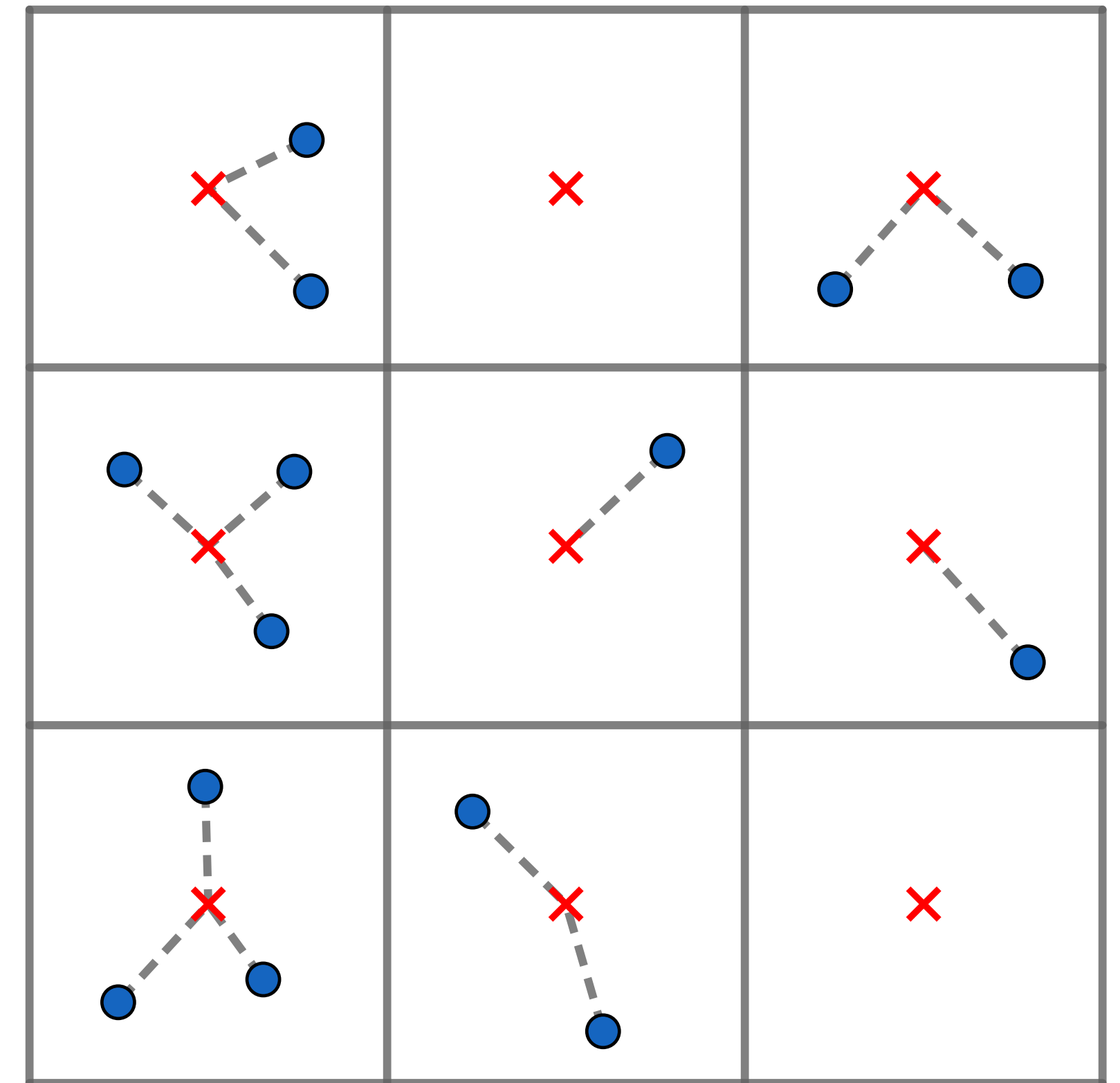
每个正方形对应x、y坐标在长度为 ℓ 区间的点： $(x, x + \ell] \times (y, y + \ell]$

- 将整个空间划分成 $\ell \times \ell$ 的小正方形
- 将数据点“round”到所属格子的格点中心
 - 由于正方形互不相交，这个rounding是唯一的

d维是 $\sqrt{d}\ell$

- 性质：每个点移动距离 $\leq \sqrt{2}\ell$

- 性质：在一个 $R \times R$ 的区域内，有至多 $\left(\frac{R}{\ell}\right)^2$ 个 $\ell \times \ell$ 正方形

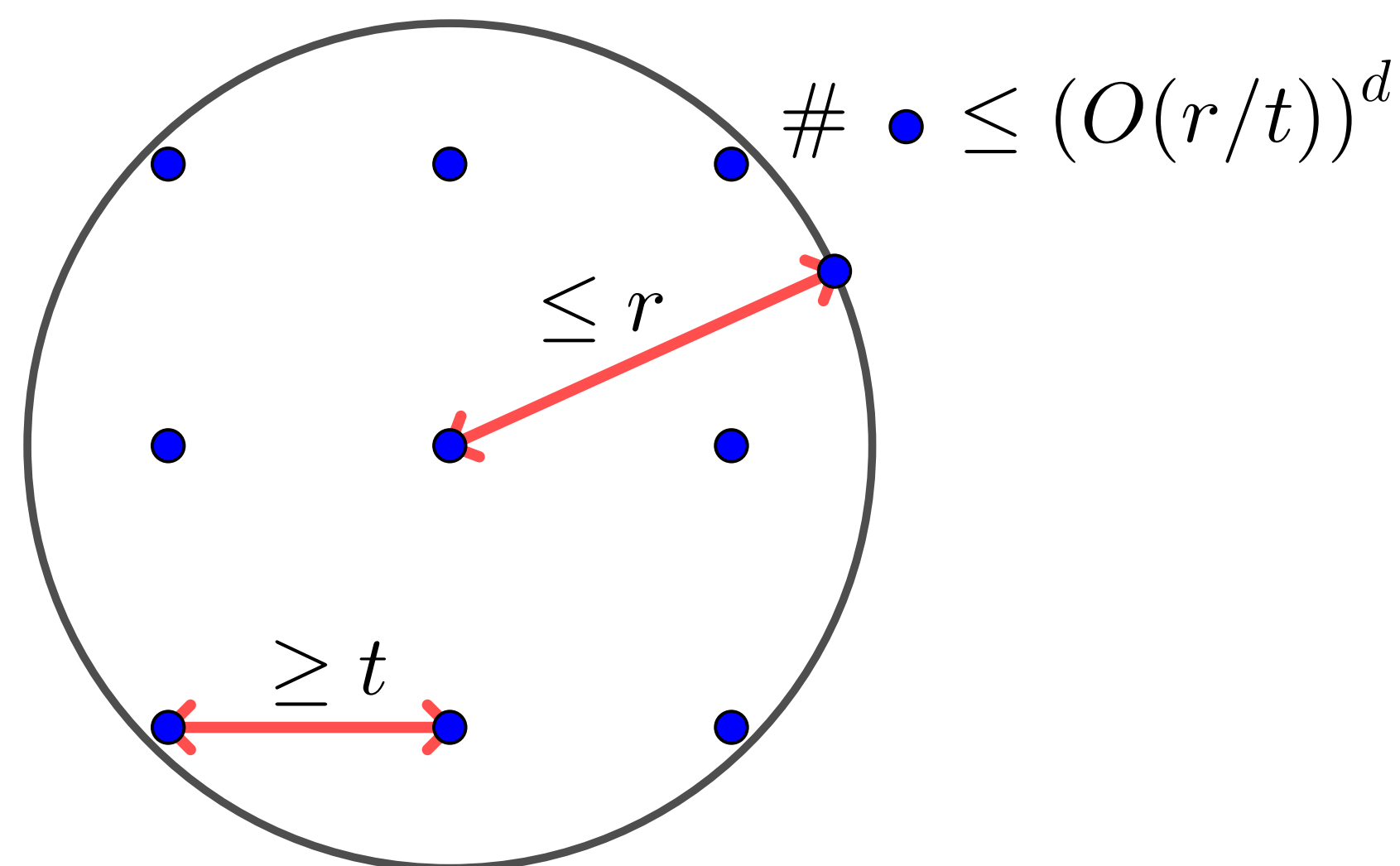
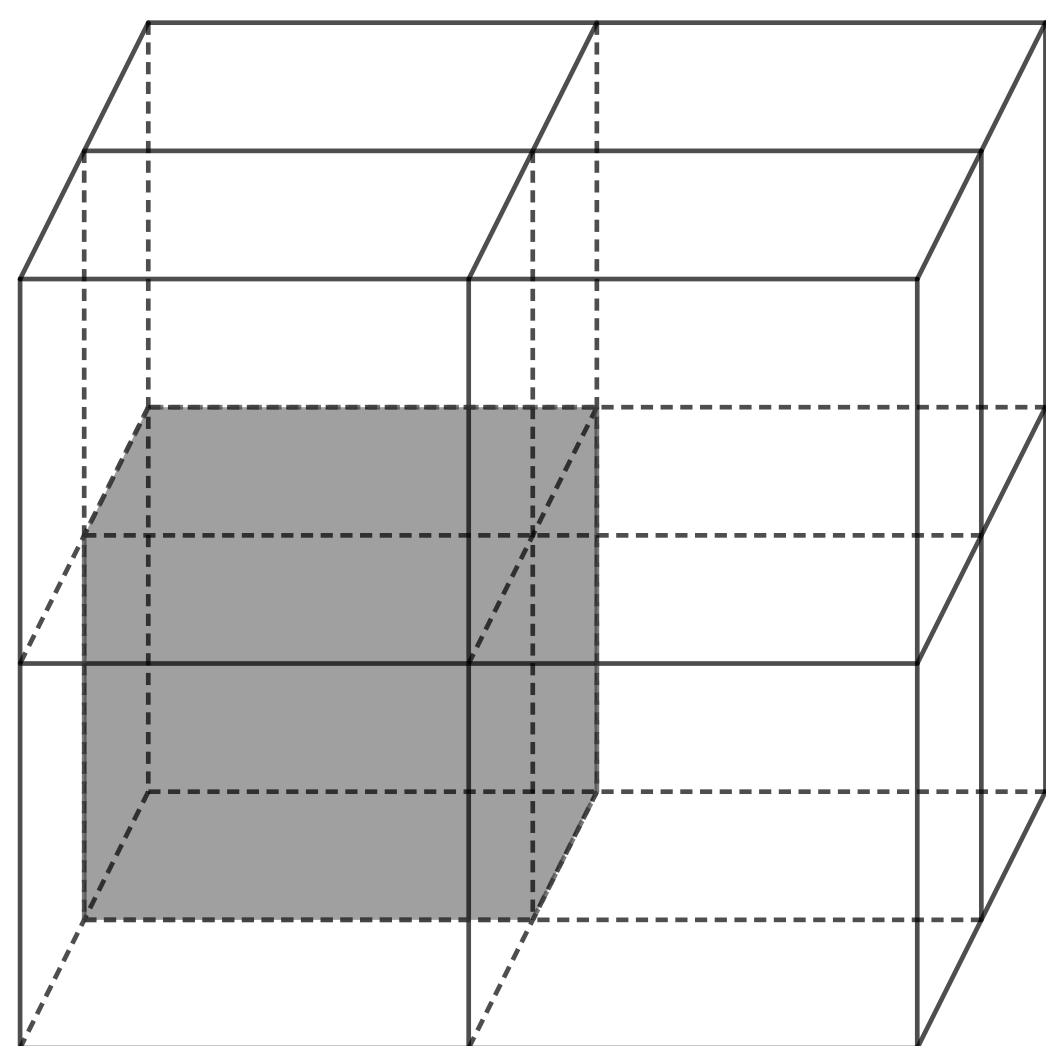


针对d维hypercube是 $(\lceil R/\ell \rceil)^d$

欧氏空间基于体积的根本性质

性质：在一个 $\overbrace{R \times \dots \times R}^d$ 的区域内，有至多 $\left(\frac{R}{l}\right)^d$ 个 $\overbrace{\ell \times \dots \times \ell}^d$ 正方形

性质：在一个半径是 r 的 d 维球里，最多可以存在 $(O(r/t))^d$ 个两两距离 $\geq t$ 的点



基本问题：直径

- 给定一个点集 $P \subseteq [\Delta]^2$
- 定义直径 $\text{diam}(P) := \max_{x,y \in P} \|x - y\|_2$
- 求 $\text{diam}(P)$

diameter

线性时间 $(1 + \epsilon)$ -近似直径

T 可通过选取任意点 u , 求 u 到最远点的距离得到 (见第一讲)

即满足 $1/2 \cdot \text{diam}(P) \leq T \leq \text{diam}(P)$

- 先 $O(n)$ 时间找一个直径的2-近似值 T
- 作 $\ell := \epsilon \cdot T / \sqrt{2}$ 的网格, 并round到中心
- 因此每个点移动了 $\leq \sqrt{2}\ell \leq \epsilon \cdot \text{diam}(P)$
- 因此新点集 P' 满足

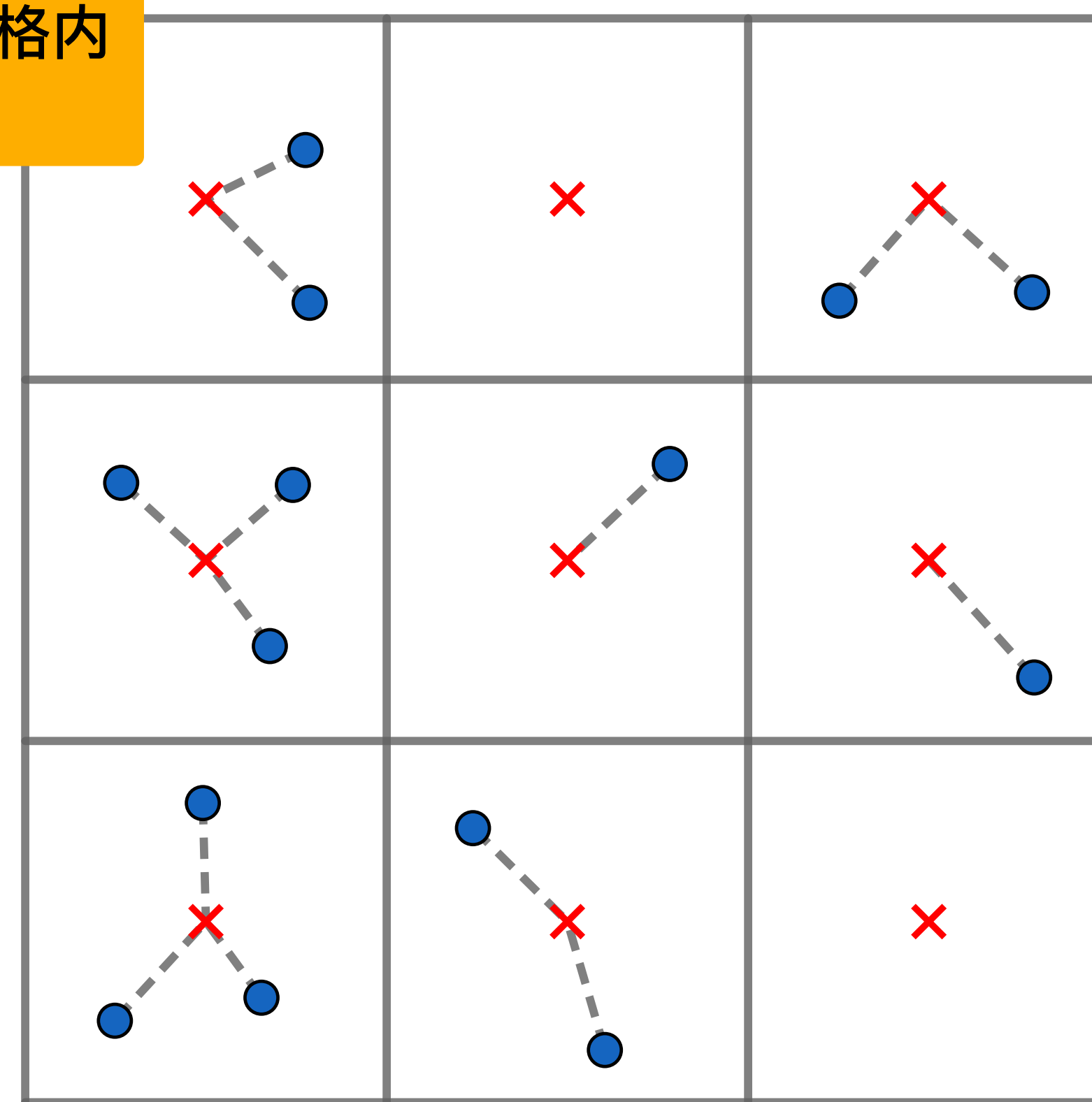
可在 $O(nd \log n)$ 时间构造 P'

$$\text{diam}(P') \in (1 \pm \epsilon) \cdot \text{diam}(P)$$

- 算法: 在 $|P'|$ 上暴力求直径, 复杂度 $|P'|^2 \cdot d$

P' 所有点都在 $\text{diam}(P') \times \text{diam}(P')$ 大方格内, 小方格 $\ell \geq \Omega(\epsilon \cdot \text{diam}(P))$, 故 $|P'| \leq (O(1/\epsilon))^2$

事实上可以是方格内任意确定点



总复杂度: $O(nd \log n) + (d/\epsilon)^{O(d)}$

一些实现细节

- 设把点都round为方格坐标较小的一角，且方格都是 $\ell \times \ell$ 的 $O(d)$ 时间
 - 那么给定 $x = (x_1, \dots, x_d)$ ，左上角的坐标是 $(\lceil x_1/\ell \rceil, \dots, \lceil x_d/\ell \rceil)$
- 直接round完后的到的 P' 会有很多的重复点，需要去重
 - 例如：排序 — $O(nd \log n)$ 复杂度
 - 排序后，值相等的元素形成连续的一段，可以 $O(n)$ 时间扫描找到所有段

作业十一： 近似求欧氏点集直径

- <http://cssyb.openjudge.cn/26hw11/>
- 分值： 1分
- Deadline: 5月6日

精确算法（但不支持动态/并行等扩展）

Welzl's Algorithm (1991)

- 相关问题：最小包围球
- 存在一个期望 $O((d + 1)(d + 1)!n)$ 时间的精确解算法
- 核心观察：设 C_i 为前 i 个点的MEB，若 $p_i \notin C_{i-1}$ 则必有 $p_i \in C_i$
- 如何利用观察：尝试求 C_{i-1} ，然后若 $p_i \notin C_{i-1}$ ，递归求包含 p_i 的前 $i - 1$ 点MEB
 - “包含 p_i ”的MEB可以递归求解
 - 最后如果边界上要求必须包括的点到达 $d + 1$ 则可以停止递归

Welzl's Algorithm

$\text{mcb}(P, R)$

if $P = \emptyset$ or $|R| = d + 1$ **return** the ball with R on the boundary

else

如果没有随机性而是任选，
算法可以是 n^d 复杂性

choose $p \in P$ u.a.r., let $D := \text{mcb}(P \setminus \{p\}, R)$

if $p \notin D$ **return** $\text{mcb}(P \setminus \{p\}, R \cup \{p\})$

return D

复杂度递推式：
$$f(n, t) \leq f(n - 1, t) + \frac{d + 1 - t}{n} f(n - 1, t + 1) + O(d)$$

解：
$$f(n, t) \leq O(n(d + 1 - t)!(d + 1 - t))$$

高维?

- 存在一个 $O(nd/\epsilon)$ 时间的 $1 + \epsilon$ 近似算法
- Smaller core-sets for balls, Badoiu and Clarkson, SODA 03

2. 递归格点离散化：四分树

递归划分：四分树

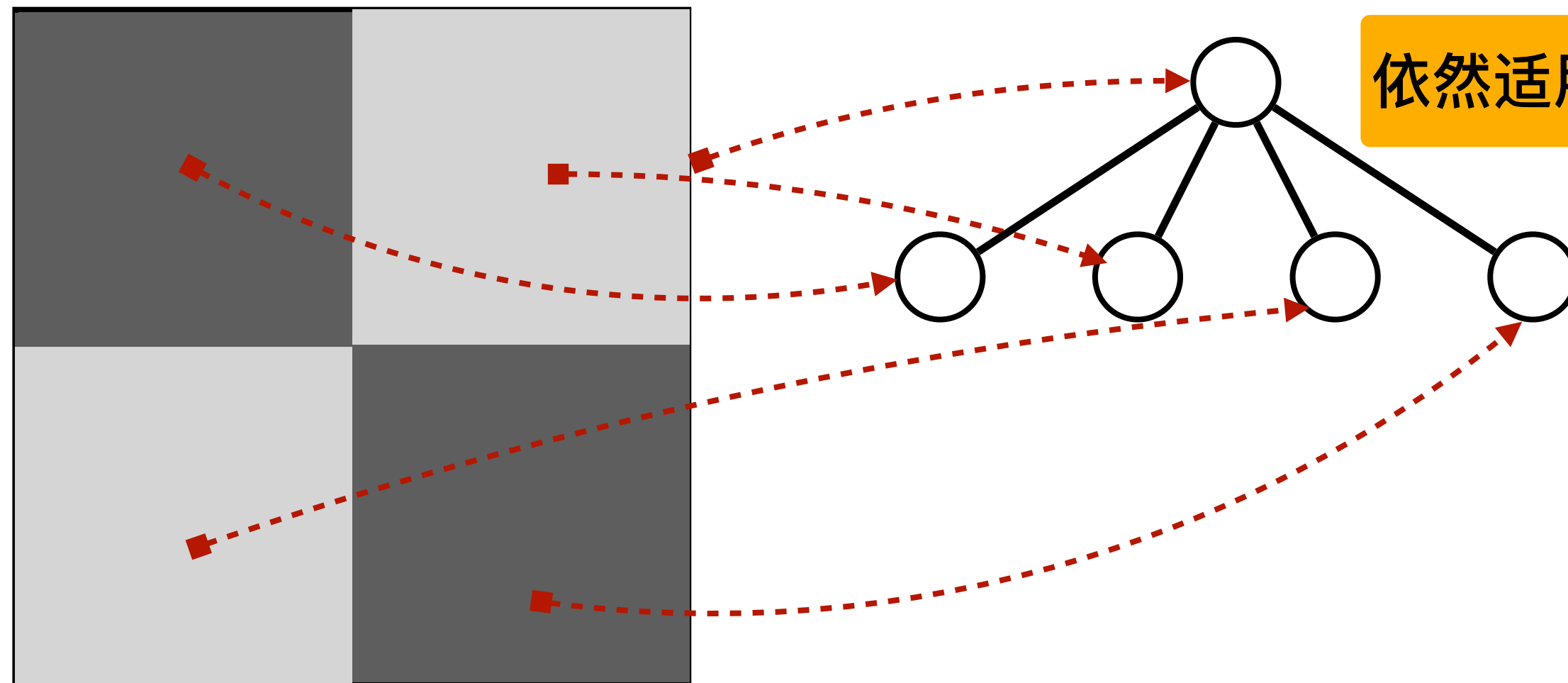
以二维为例

d维叫“d维四分树”

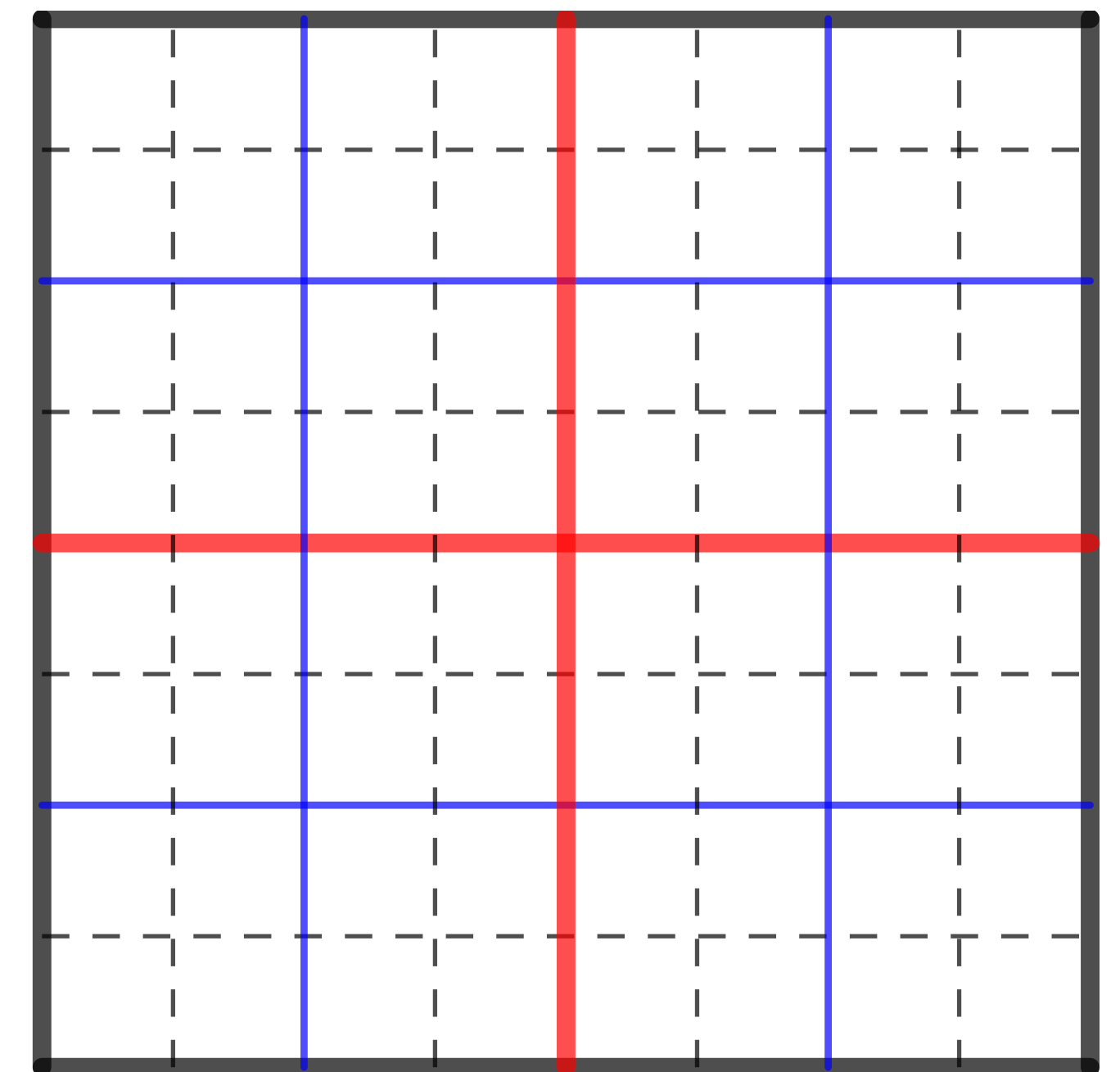
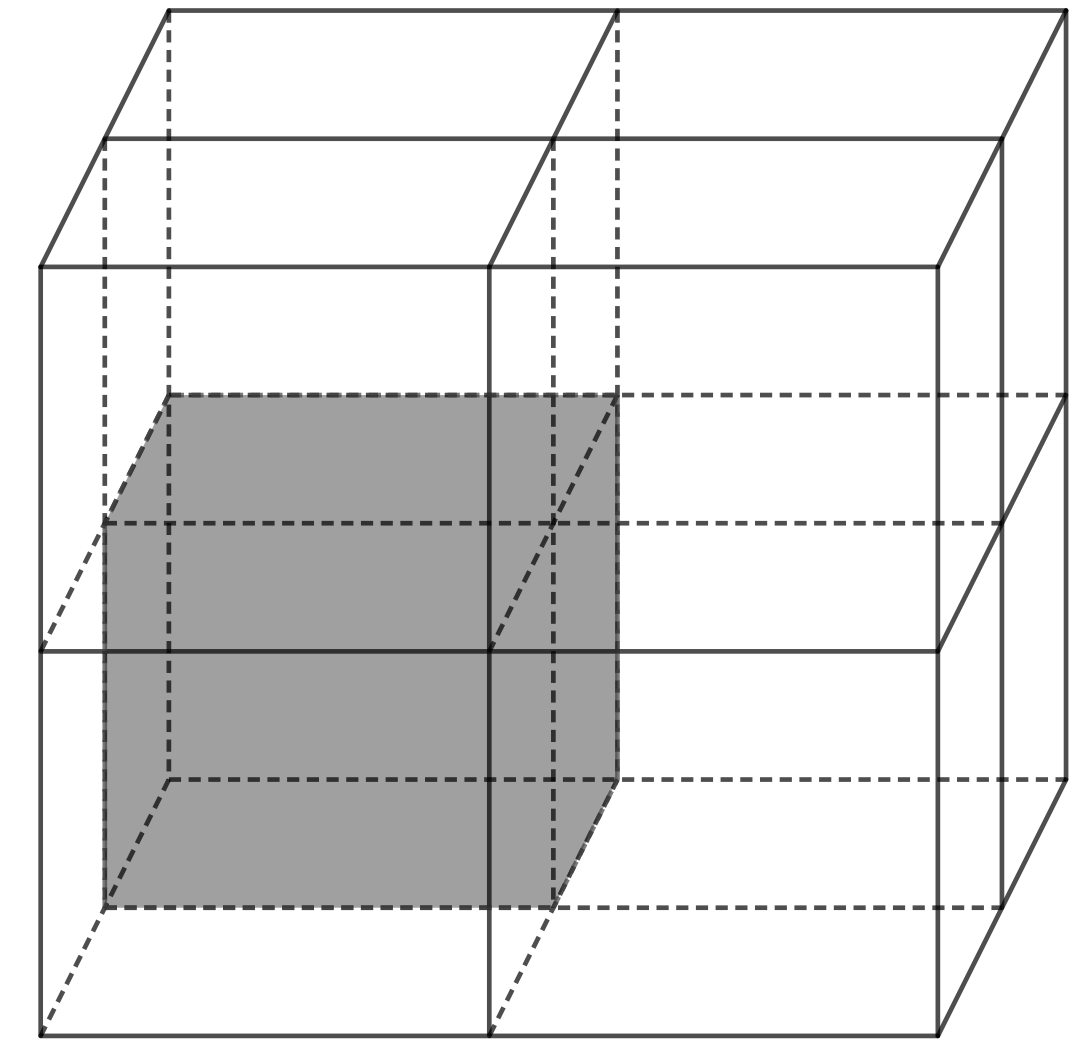
d维是从 $[\Delta]^d$ 划分

- 从最大的 $\Delta \times \Delta$ 的正方形开始
- 递归将当前 $\square$ 等分成4个边长为一半的 $\square$
- 组织成树形结构，直到只含单点停止，至多 $\log_2 \Delta$ 层

d维分成 2^d 个边长为一半的 $\square$



依然适用于d维



伪代码

d维

全局调用 $\square = [\Delta]^d$

BuildTree($\square$, P)

两个停止情况：不含数据点直接返回空；只含1个点返回当前 $\square$

if ($P \cap \square = \emptyset$) return NULL

let T.root = ($\square$, $\square \cap P$)

除了记录 $\square$ ，也要记录 $\square$ 里含有的数据点集

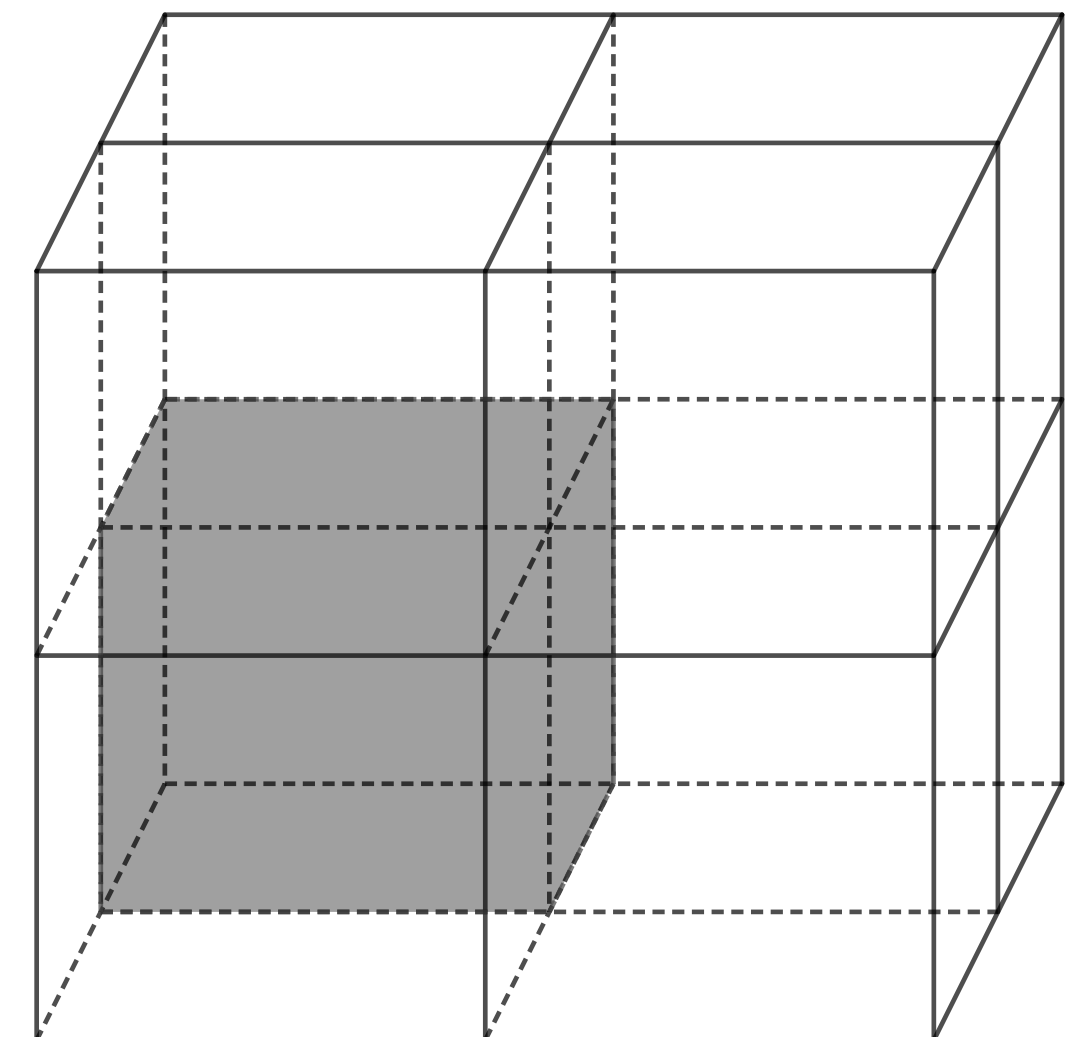
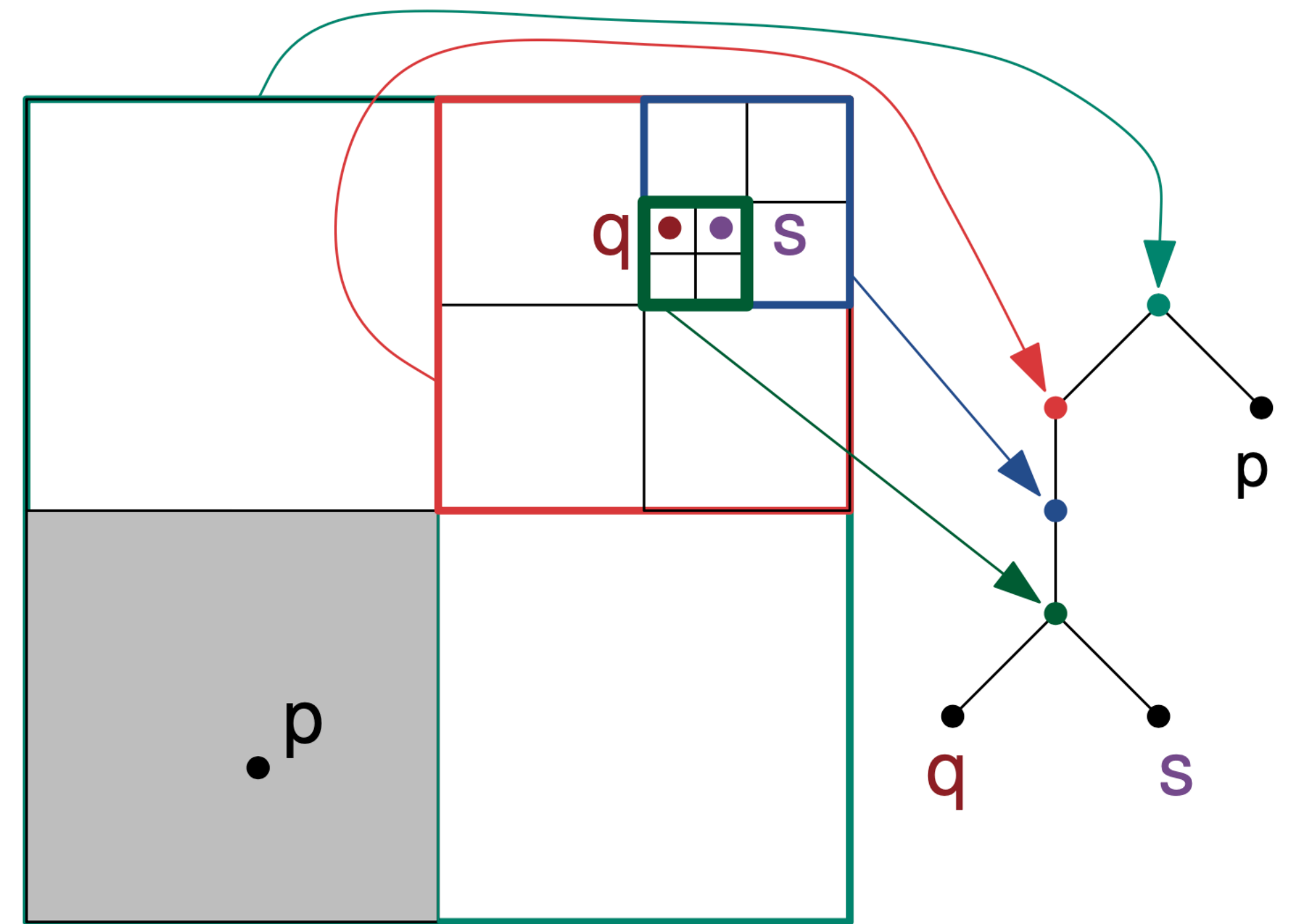
if ($|P \cap \square| = 1$) return T

evenly divide $\square$ into 2^d sub-squares $\square_1, \dots, \square_{2^d}$

for $i = 1, \dots, 2^d$, let T.child $_i$ = BuildTree($\square_i$, $P \cap \square_i$)

return T

整个算法只会产生 $O(n \log \Delta)$ 个非空 $\square$ (非空即 $\square \cap P \neq \emptyset$)



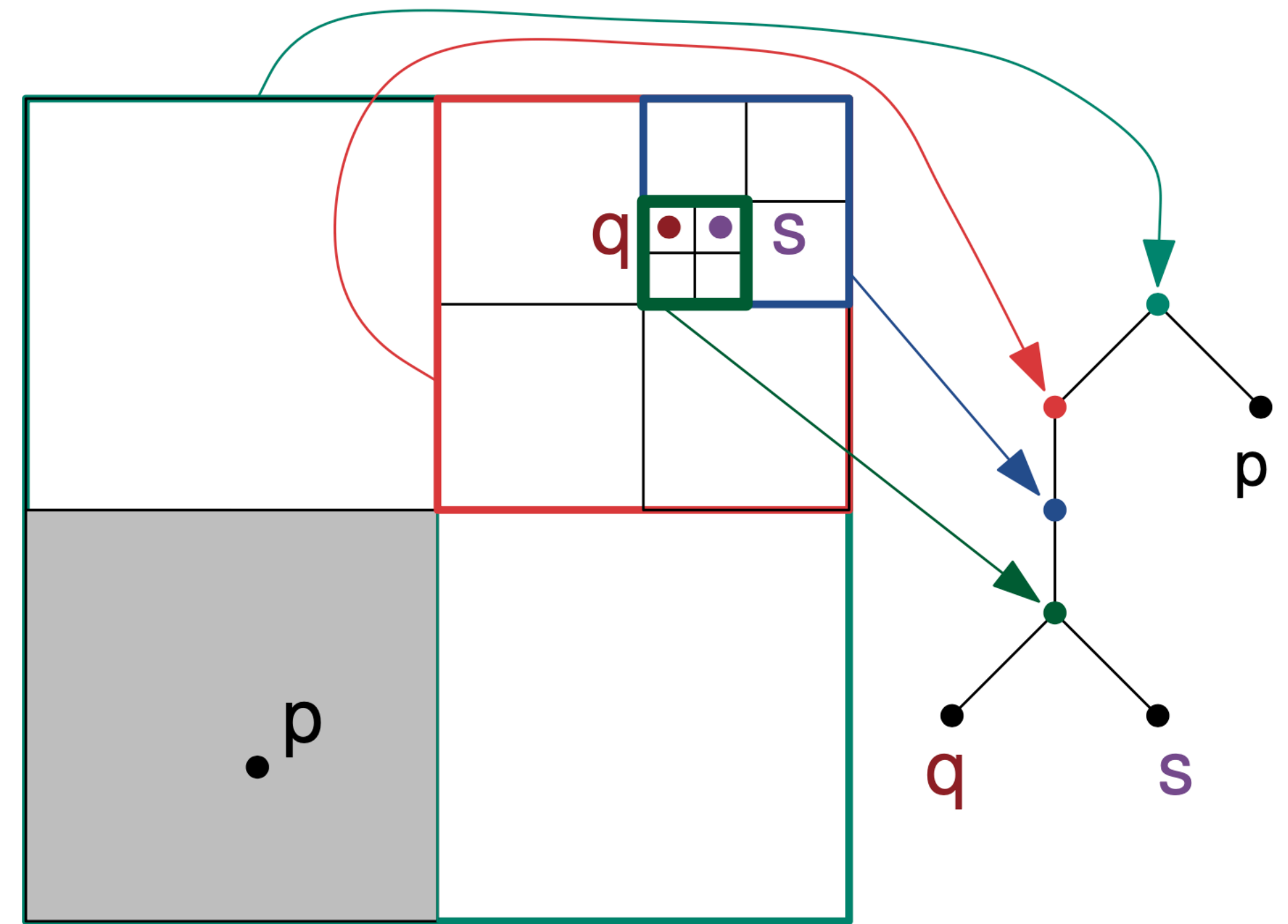
实现细节

存储

- 每个 $\square$ 对应一个树上的节点
- 另需开一个数组/vector<int>来存 $P \cap \square$
- 还需要存储 2^d 个 $\square_i$ 对应的指针
- 总空间复杂度: $O(n \cdot 2^d \log \Delta)$
- 树的每层 $\square$ 的并集存储整个点集 P
- 共 (至多) $\log \Delta$ 层

2^d 来自于存 2^d 个 $\square_i$ 对应指针所需空间

注意: 该树中没有空 $\square$



实现细节

可选优化：省去 2^d

evenly divide $\square$ into 2^d sub-squares $\square_1, \dots, \square_{2^d}$

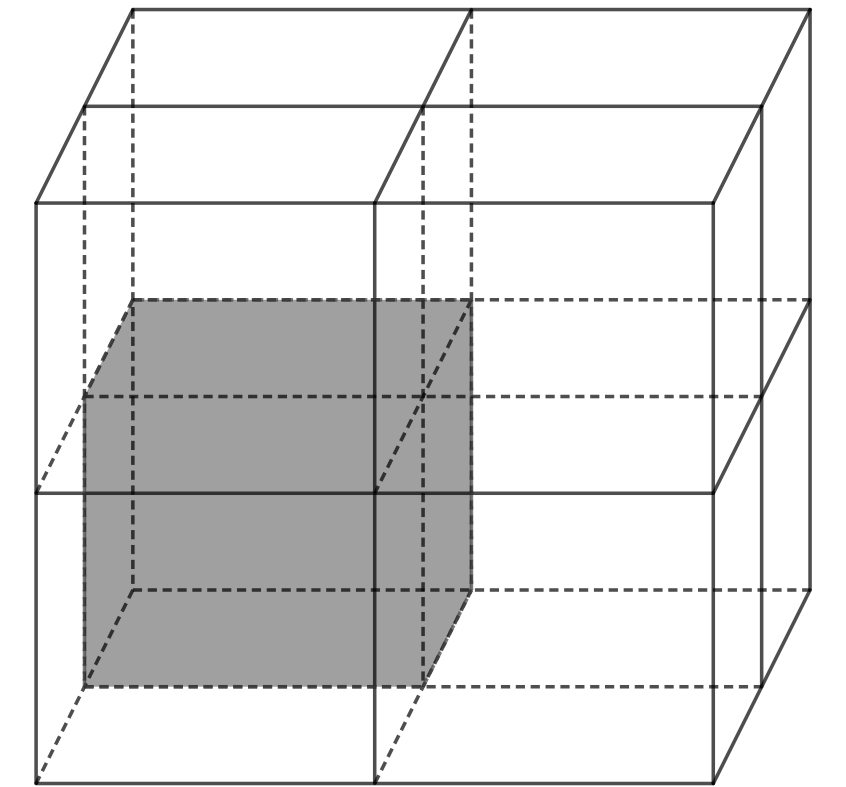
for $i = 1, \dots, 2^d$, let $T.\text{child}_i = \text{BuildTree}(\square_i, P \cap \square_i)$

- 这两行直接实现需要对每个树中节点穷举 2^d 个 $\square_i$ ，共需 $2^d \cdot n \log \Delta$ 时间和空间
- 接着上页，把主循环检查点集 P ，只保留非空的 $\square_i$ ，这样就是 $O(|P|d)$ 时空
 - 所有层总共就是 $nd \log \Delta$
- 总时空复杂度： $O(nd \log \Delta)$ 若使用该优化， 否则是 $O(n2^d \log \Delta)$

不采用该优化会使算法简单一些，若 d 没有很大，可以不采用

实现细节

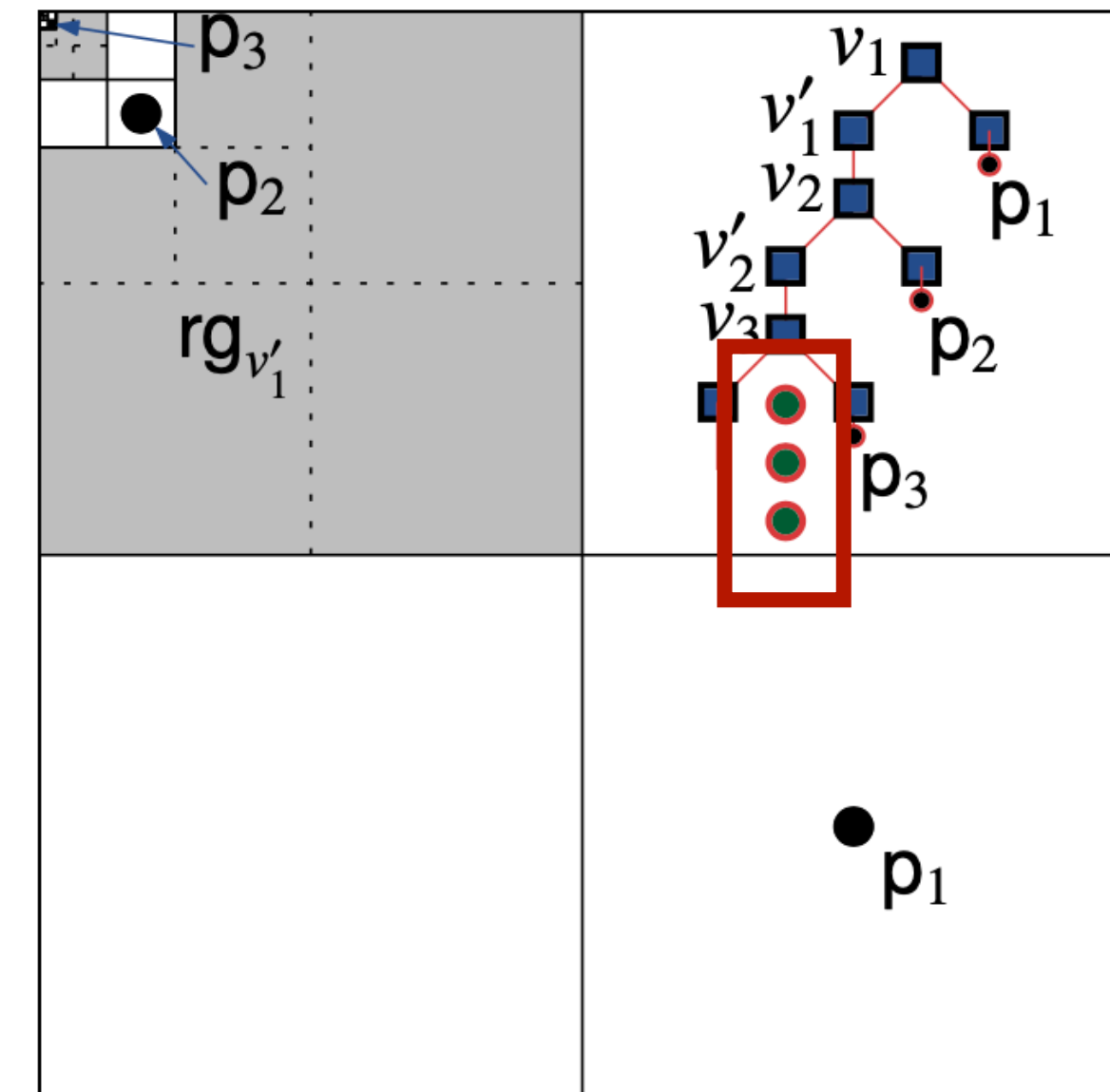
对 $i = 1, \dots, 2^d$ 求 $P \cap \square_i$ 总共需要多少时间?



- 暴力: $n \cdot 2^d \cdot d$, 即所有 $x \in P$ 与 $\square_i$ 比较一次 (每次比较代价是 d)
- 可以做到 $n \cdot d$:
 - 设当前 $\square = [\ell] \times [\ell]$, 则 2^d 个 $\square_i$ 是每维从 $(0, \ell/2]$ 和 $(\ell/2, \ell]$ 2选1进行组合
 - $\square_i$ 每维选哪个区间可用 i 的二进制表示来确定
- 算法:
 - 对 $x \in P$, 判断 x 每一维属于 $(0, \ell/2]$ 还是 $(\ell/2, \ell]$, $O(d)$ 时间确定所属 $\square_i$

其他改进： 压缩四分树

- 我们讨论的四分树有 $O(n \log \Delta)$ 个 $\square$
- 我们可以将四分树“压缩”，使得只有 $O(n)$ 个 $\square$
- $\log \Delta$ 哪来的？ 如何压缩？
 - 树中度 = 2 的点构成一个长 $O(\log \Delta)$ 路径
 - 具体： 设路径起点 $\square_u$ 终点 $\square_v$ ，则设置 $\square_u$ 的唯一孩子为 $\square_v$ ，中间全都删除



利用四分树进行 ϵ -近似最近邻查询

- 问题：

- 对输入点集 $P \subseteq [\Delta]^d$ 进行预处理

即 $\hat{q}$ 到 q 的距离是 q 到 q 在 P 最近邻距离的 $(1 + \epsilon)$ 倍

- 对任意 $q \in [\Delta]^d$ 回答 $\hat{q} \in P$, 使得 $\|\hat{q} - q\| \leq (1 + \epsilon) \cdot \min_{q' \in P} \|q' - q\|$

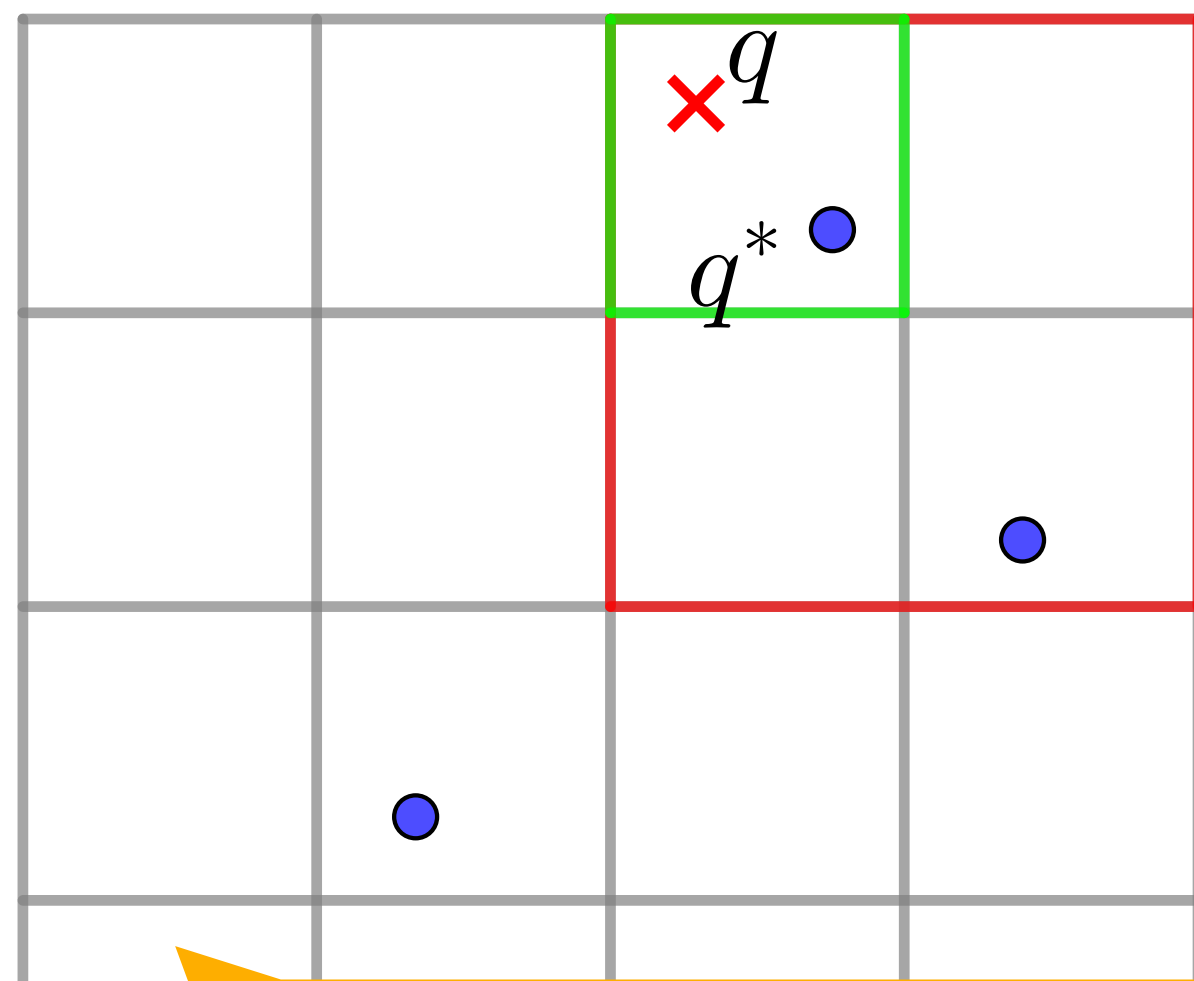
- 预处理：BuildTree($\square$, P), 这里调用 $\square = [\Delta]^d$

- 我们将给出一个 $O(\epsilon^{-d} \log \Delta)$ 时间的查询算法返回满足要求的 $\hat{q}$

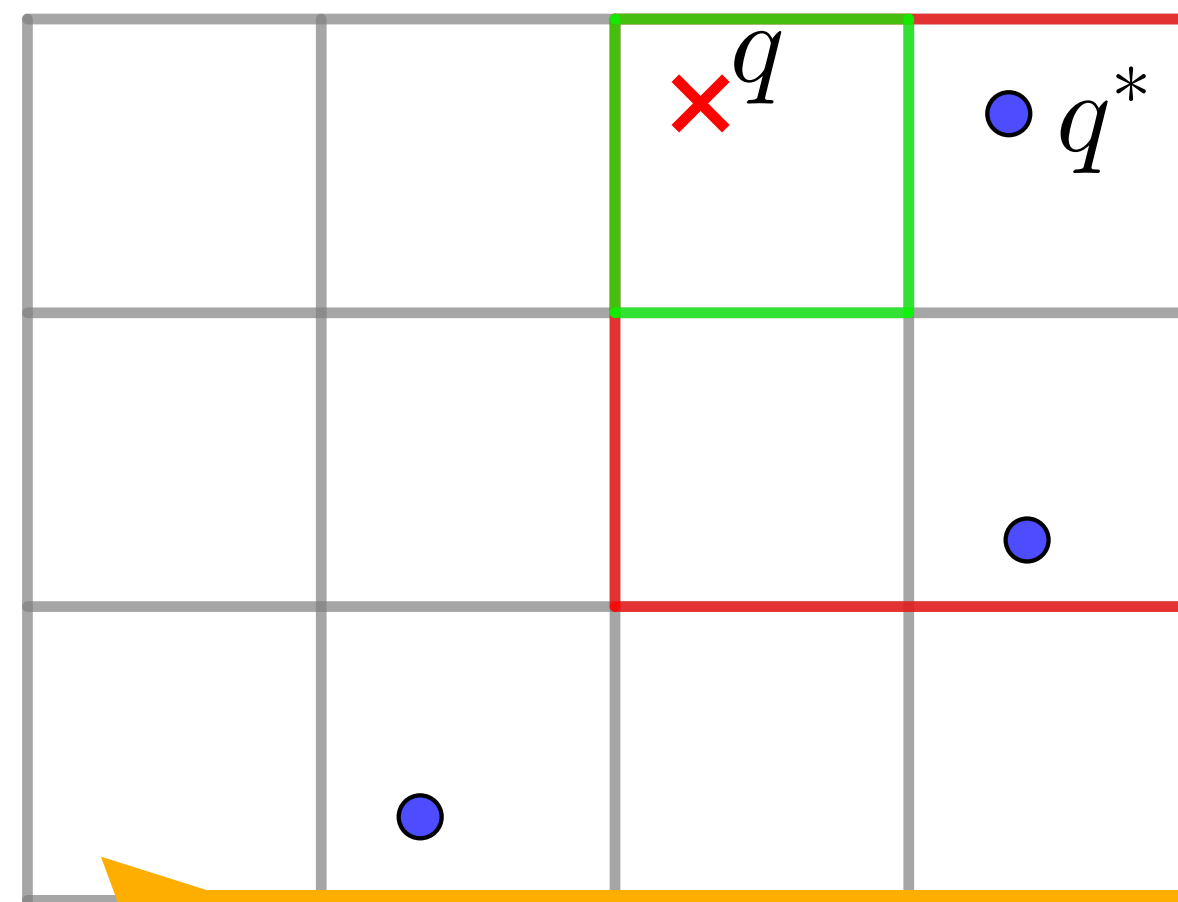
查询：一个（错误的）理想化算法

- 设查询点是 q ，最近邻是 q^* ，距离是 $r^* = \|q - q^*\|$
- 从根 $\square$ 开始，每次递归进入 q 所在的某个 $\square_i$ ，直到当前 $\square$ 只有一个点 q' ，返回 q'

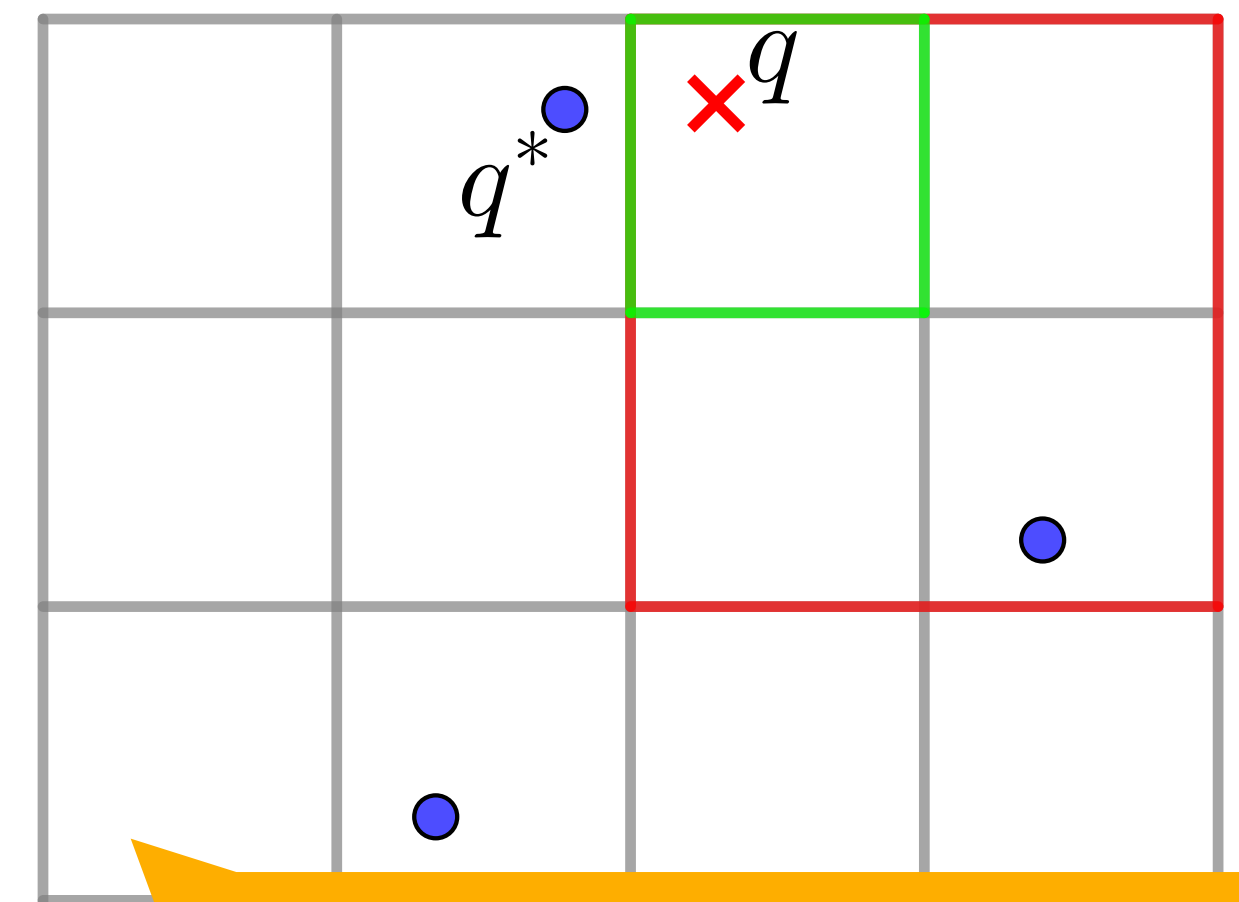
这对应于沿着 $\log \Delta$ 层的路径走到底



成功的情况：经历红方格-绿方格，最后找到了 q^*



问题：绿方格可能是空的，只能从红方格找



问题： q^* 甚至可能不在红格子里，因此不能只递归查找 q 所在格子，周围格子也要考虑

正确的算法？

- 正确的算法不能只从q所在格子递归
 - 递归搜索的时候，需要把全部 2^d 个 $\square_i$ 都检查一下
- 但这样复杂度无保证.....
- 为优化复杂度，需要一些“剪枝”，即排除掉一些“过远”的 $\square_i$

剪枝策略

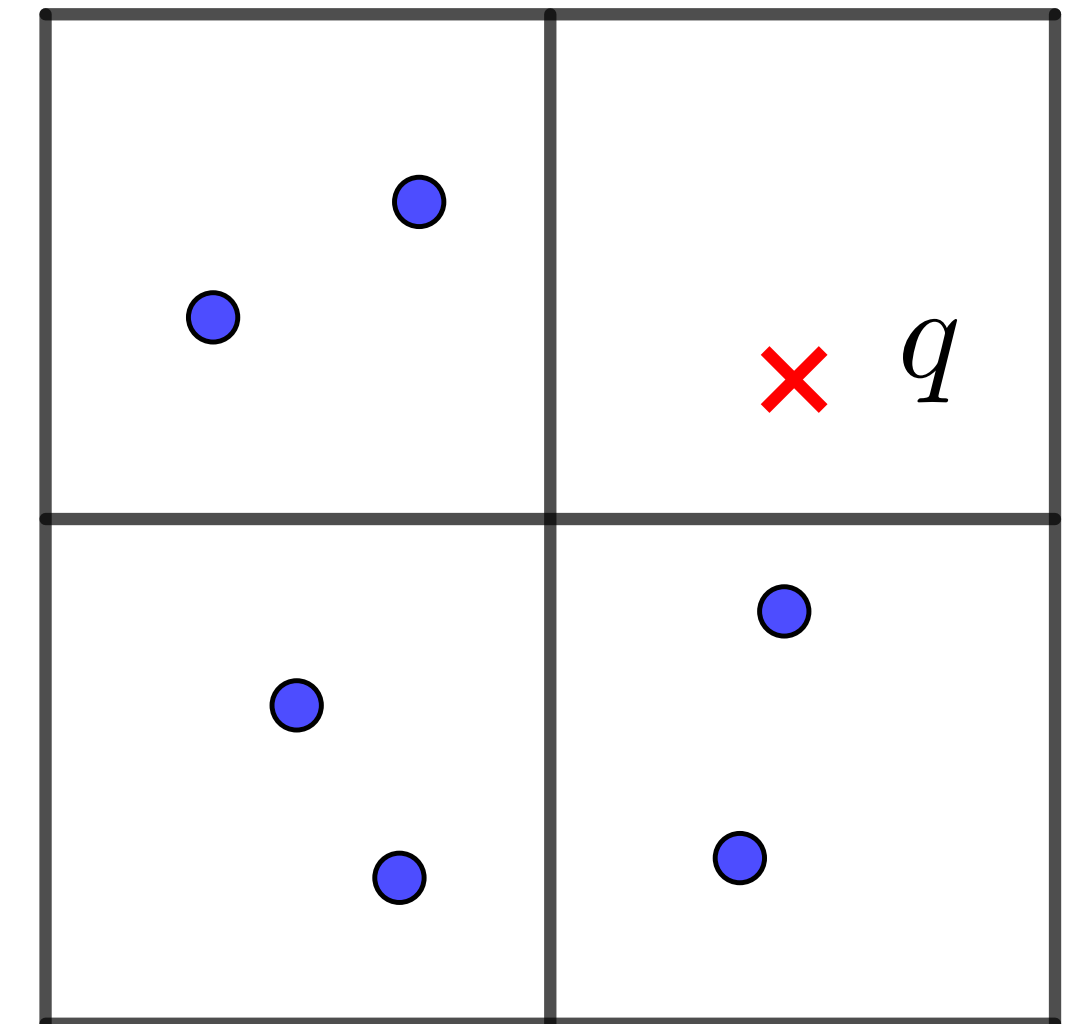
- 以第一层的4个 $\square_i$ 为例：什么情况下才不用在 $\square_i$ 继续递归？
- 根本要求：不能miss掉最近邻 q^* 的 $(1 + \epsilon)$ -近似
- 设搜索过程中维护一个当前最优解 $\hat{q}$ ，设 $r := \|q - \hat{q}\|$
- 剪枝策略：若对任何 $q' \in \square_i$ 都有

$$r \leq (1 + \epsilon) \cdot \|q - q'\|$$

则即使 $q' = q^*$ 也不再需要考虑 $\square_i$ 了

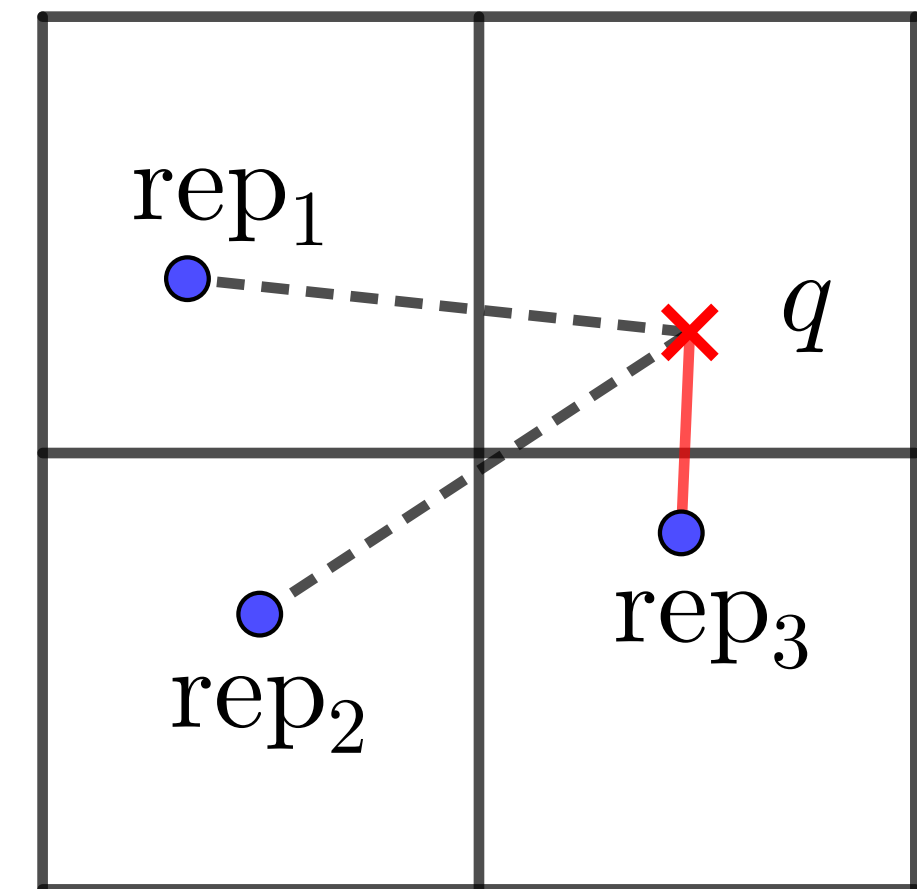
但这个判据很难高效实现

说明当前解是 $\square_i$ 中任何点的 $(1 + \epsilon)$ -近似



进一步离散化：选取代表点

- 优化策略：每个□任选一个数据点作为代表点，记为 $\text{rep}_\square$
- 为优化复杂度，算法只会从rep中搜索（近似）最近点



新的剪枝规则

- 因为现在只考虑代表点 rep ，剪枝规则也需要跟着改
- 依然设 r 为“当前最优解” $\hat{q}$ 到查询点 q 的距离
- 新剪枝： $\square_i$ 不需要继续考虑，若

$$r \leq (1 + \epsilon) \cdot (\|q - \text{rep}_i\| - \text{diam}(\square_i))$$

$\square_i$ 的直径； $\ell \times \ell$ 的是 $\sqrt{2}\ell$

将 $\leq$ 替换成 $>$ 则可得到“需要继续考虑 $\square_i$ ”的判据

新旧剪枝条件的对比

- **旧剪枝**： $\square_i$ 不需被考虑，若 $\forall p \in \square_i$ ，有 $r \leq (1 + \epsilon) \cdot \|q - p\|$
- **新剪枝**： $\square_i$ 不需被考虑，若 $r \leq (1 + \epsilon)(\|q - \text{rep}_i\| - \text{diam}(\square_i))$
- 新剪枝条件更难以满足（即更不会误剪）：
- 即某个 $\square_i$ 在新剪枝剪掉 $\Rightarrow$ $\square_i$ 在旧剪枝剪掉：
- 需要验证：若 $r \leq (1 + \epsilon)(\|q - \text{rep}_i\| - \text{diam}(\square_i))$ ，则

$$\forall p \in \square_i, \text{ 有 } r \leq (1 + \epsilon) \cdot \|q - p\|$$

进一步解释

- 现验证：若 $r \leq (1 + \epsilon)(\|q - \text{rep}_i\| - \text{diam}(\square_i))$ ，则

$$\forall p \in \square_i, \text{ 有 } r \leq (1 + \epsilon) \cdot \|q - p\|$$

- 性质： $\forall p \in \square_i$ ，有 $\|q - \text{rep}_i\| - \text{diam}(\square_i) \leq \|q - p\|$

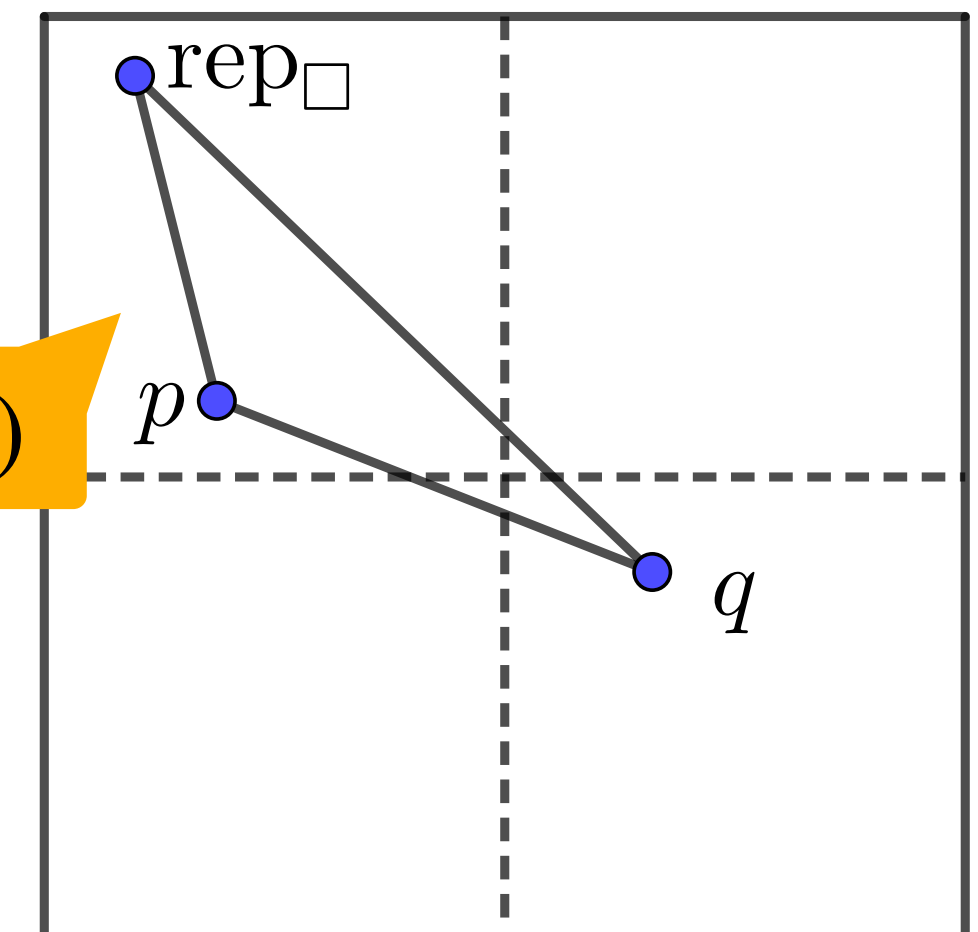
- 观察：任何 p 可由 rep_i 移动至多 $\text{diam}(\square_i)$ 距离到达

- 数学语言：

$$\|q - p\| \geq \|q - \text{rep}_i\| - \|\text{rep}_i - p\| \geq \|q - \text{rep}_i\| - \text{diam}(\square_i)$$

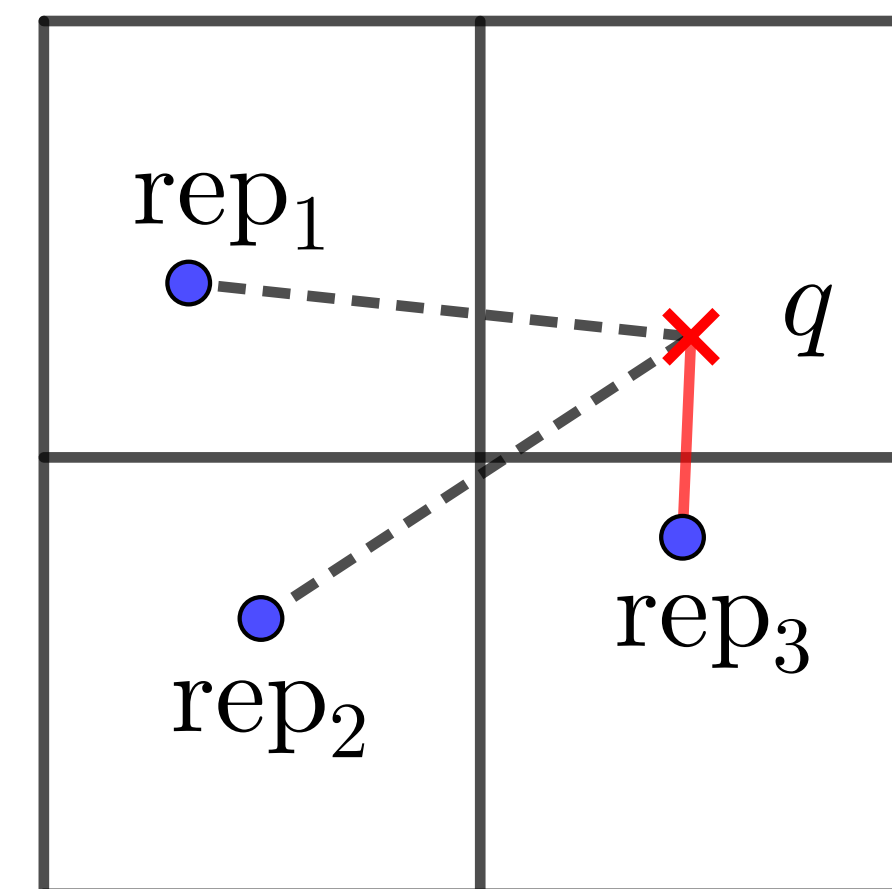
三角形不等式

利用 $\|\text{rep}_i - p\| \leq \text{diam}(\square_i)$



完整算法

输入：查询点 q ，误差参数 $0 < \epsilon < 1$ ；查询所需时间 $O(\epsilon^{-d} \log \Delta)$



设集合 A_i 为需要检查的四分树第 i 层的格子的集合 ($0 \leq i \leq \log \Delta$)

初始 A_0 放入最大格子 $\square = [\Delta]^d$, $A_i = \emptyset$ ($1 \leq i \leq \log \Delta$), $r = \infty$, $\hat{q} = \text{NULL}$

for $i = 1, \dots, \log \Delta$

i 是层数，按层进行迭代

$\hat{q}$ 是当前解， r 是 $\|q - \hat{q}\|$

设 S 为所有 A_{i-1} 的格子的 2^d 个子 $\square$ 的集合

更新当前解为本层最近的rep

对每个 $\square_w \in S$ ，若 $\|q - \text{rep}_w\| < r$ ，更新 $r = \|q - \text{rep}_w\|$ 及 $\hat{q} = \text{rep}_w$

对每个 $\square_w \in S$ ，若 $r > (1 + \epsilon) \cdot (\|q - \text{rep}_w\| - \text{diam}(\square_w))$ ，将 $\square_w$ 放入 A_i

此处是“需要放入 $\square_w$ ”的判据，因此是 $>$

返回 $\hat{q}$

作业十二：欧氏空间近似最近邻查询

- <http://cssyb.openjudge.cn/26hw12/>
- 分值：2分
- Deadline: 5月8日

* 算法分析

- 我们要说明的：
 - **正确性**：算法结束时找到的点满足 $r \leq (1 + \epsilon)r^*$
 - **效率**：总运行时间是 $O(\epsilon^{-O(d)} \log \Delta)$
- 先给出两个性质，然后推出 正确性 和 效率 的分析

*** 性质1: 若 $r > (1 + \epsilon)r^*$ 则 q^* 所在 $\square$ 不会被剪枝**

“算法在当前解依然很坏的时候, 从来不会miss掉最优解”

- 回忆算法步骤:

即剪枝规则

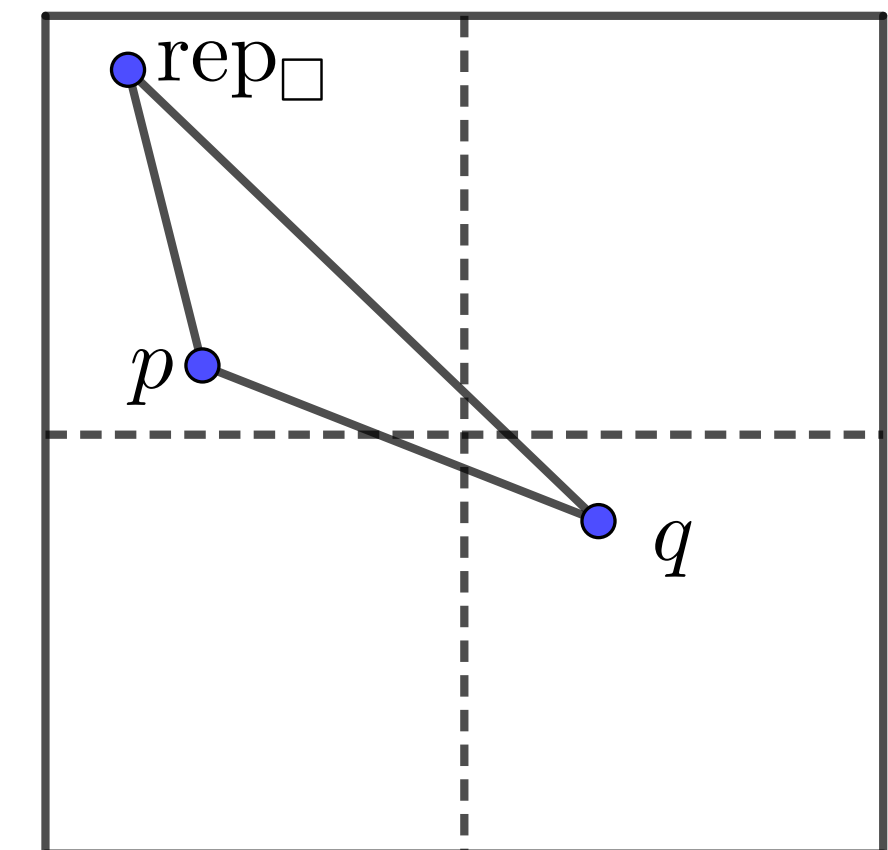
对每个 $\square_w \in S$, 若 $r > (1 + \epsilon) \cdot (\|q - \text{rep}_w\| - \text{diam}(\square_w))$, 将 $\square_w$ 放入 A_i

- 设 $\square_w$ 为 q^* 所在格子, 下面验证 $\square_w$ 满足放入 A_i 的判定条件

Claim: 设当前在第 i 层且 $r > (1 + \epsilon)r^*$, 则 q^* 所在格子一定可以放入 A_i

$$\begin{aligned} & (1 + \epsilon) \cdot (\|q - \text{rep}_w\| - \text{diam}(\square_w)) \\ & \leq (1 + \epsilon)\|q - q^*\| \\ & = (1 + \epsilon) \cdot r^* \\ & < r \end{aligned}$$

利用性质: $\forall p \in \square_i$, 有
 $\|q - \text{rep}_i\| - \text{diam}(\square_i) \geq \|q - p\|$



* 性质2: 算法运行到 $\text{diam}(\square) \leq \epsilon/2 \cdot r^*$ 就会停止

“算法走到四分树足够深的地方就会停止, 从而不继续访问不再需要访问的格子”

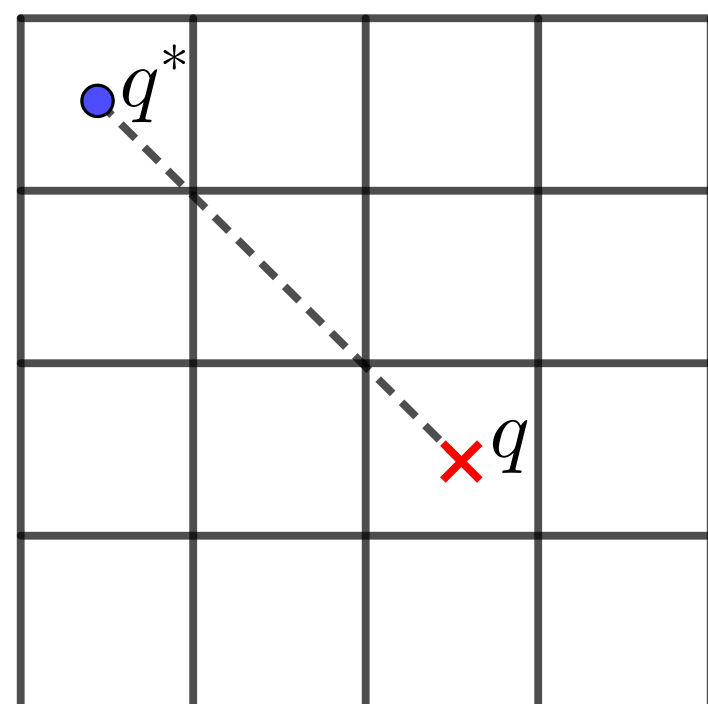
• 回忆算法步骤:

(*) 对每个 $\square_w \in S$, 若 $\|q - \text{rep}_w\| < r$, 更新 $r = \|q - \text{rep}_w\|$ 及 $\hat{q} = \text{rep}_w$

(**) 对每个 $\square_w \in S$, 若 $r > (1 + \epsilon) \cdot (\|q - \text{rep}_w\| - \text{diam}(\square_w))$, 将 $\square_w$ 放入 A_i

• Claim: 若 $\text{diam}(\square_w) \leq \epsilon/2 \cdot r^*$, 则 $|A_i| = \emptyset$

即验证: 任何 $\square_w \in S$ 都被剪枝, 无法放入 A_i



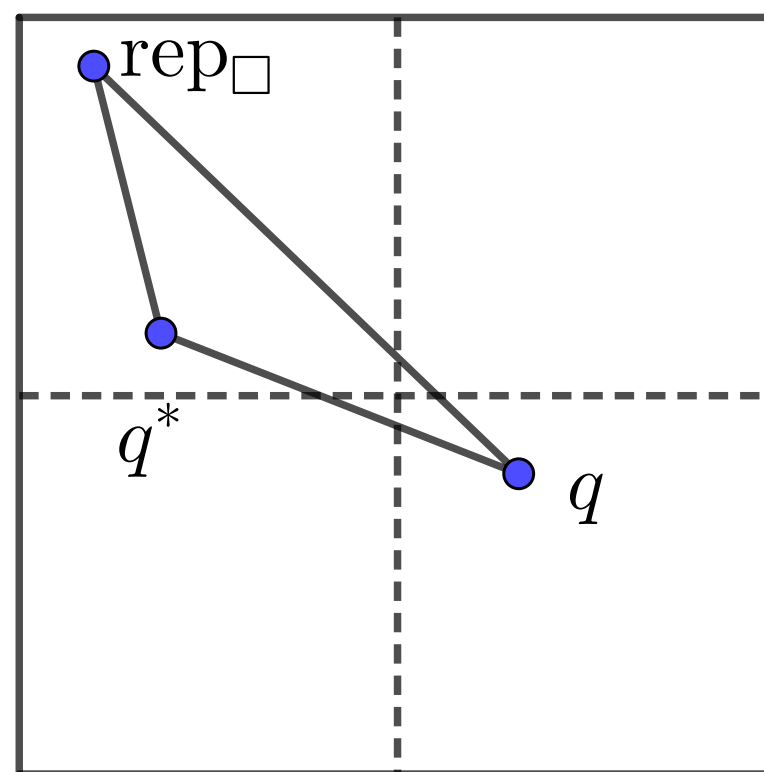
直觉: $\text{diam}(\square)$ 很小
($\leq O(\epsilon) \cdot r^*$) 说明近似的
足够精确了, 没有任何点加
进去可以显著改进精度了

$$\begin{aligned} & (1 + \epsilon) \cdot (\|q - \text{rep}_w\| - \text{diam}(\square_w)) \\ & \geq (1 + \epsilon) \cdot (r - \text{diam}(\square_w)) \quad \text{利用(*)} \\ & \geq (1 + \epsilon) \cdot (r - \epsilon/2 \cdot r^*) \\ & \geq (1 + \epsilon) \cdot (1 - \epsilon/2) \cdot r \quad \text{利用 } r^* \leq r \\ & \geq r \end{aligned}$$

* 正确性分析

总是能找到 $r \leq (1 + \epsilon)r^*$

- 因为算法在 $\text{diam}(\square) < \epsilon/2 \cdot r^*$ 后就停止了，只需考虑 $\text{diam}(\square) \geq \epsilon/2 \cdot r^*$
- 若停止前的某个时刻 $r \leq (1 + \epsilon) \cdot r^*$ ，则已经达到了要求
- 否则， $r > (1 + \epsilon) \cdot r^*$ ，那么 性质1 说明 q^* 所在格子依然存活
- 当 $\text{diam}(\square) \approx \epsilon/2 \cdot r^*$ ，我们有 $r \leq r^* + \text{diam}(\square) \leq (1 + \epsilon)r^*$



根据三角形不等式：

$$r \leq \|q - \text{rep}\| \leq \|q - q^*\| + \|\text{rep} - q^*\| \leq r^* + \text{diam}(\square)$$

* 运行时间分析

当 $\text{diam}(\square_w) > \epsilon/2 \cdot r^*$ 时 $|A_i|$ 的上界

- 回忆算法步骤：

(*) 对每个 $\square_w \in S$ ，若 $\|q - \text{rep}_w\| < r$ ，更新 $r = \|q - \text{rep}_w\|$ 及 $\hat{q} = \text{rep}_w$

(**) 对每个 $\square_w \in S$ ，若 $r > (1 + \epsilon) \cdot (\|q - \text{rep}_w\| - \text{diam}(\square_w))$ ，将 $\square_w$ 放入 A_i

- 若 $\square_w \in S$ 放入 A_i ，则 $\|q - \text{rep}_w\| < r/(1 + \epsilon) + \text{diam}(\square_w)$ 根据 (**)

- Claim:** $r/(1 + \epsilon) + \text{diam}(\square_w) \leq O(1/\epsilon) \cdot \text{diam}(\square_w)$

- 先假定Claim正确：则若 $\square_w$ 放入了 A_i ， $\|q - \text{rep}_w\| \leq O(1/\epsilon) \cdot \text{diam}(\square_w)$

* 推出 $|A_i|$ 上界

- 因为 $\|q - \text{rep}_w\| \leq O(1/\epsilon) \cdot \text{diam}(\square_w)$
- 所以所有放入 A_i 的 rep_w 都在以 q 为球心 $O(1/\epsilon) \cdot \text{diam}(\square_w)$ 为半径的 d 维球里

性质：在一个 $\overbrace{R \times \dots \times R}^d$ 的区域内，有至多 $\left(\frac{R}{l}\right)^d$ 个 $\overbrace{\ell \times \dots \times \ell}^d$ 正方形

每个 rep_w 对应一个正方形！

- 因此有 $|A_i| \leq (O(1/\epsilon))^d$

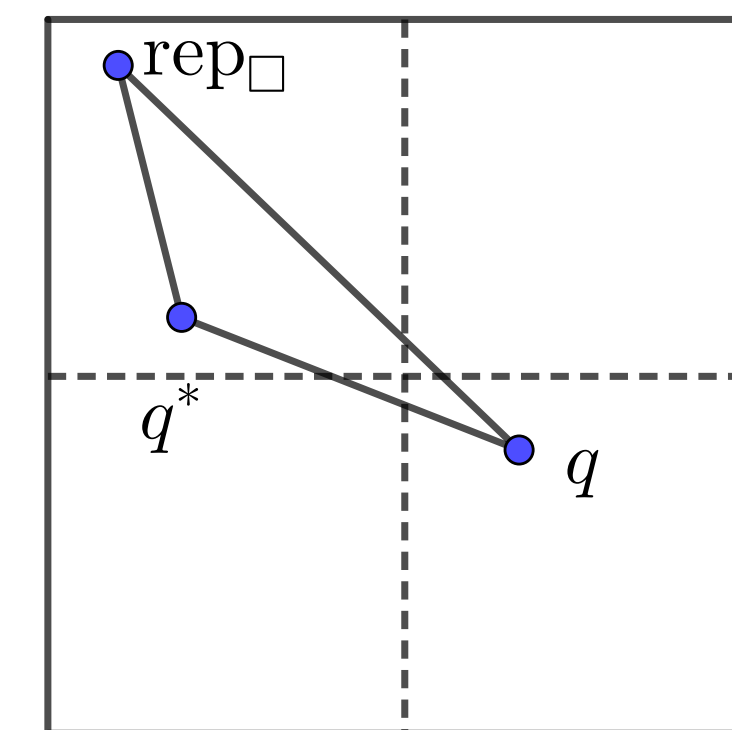
至多有 $O(\log \Delta)$ 个 $|A_i|$ 非零，因此总复杂度 $(O(1/\epsilon))^d \cdot \log \Delta$

* 验证Claim

Claim: $r/(1 + \epsilon) + \text{diam}(\square_w) \leq O(1/\epsilon) \cdot \text{diam}(\square_w)$

- 等价于证 $\text{diam}(\square_w) \geq O(\epsilon/(1 + \epsilon)) \cdot r$
根据性质2；否则会导致 $|A_i| = 0$ ，不需要考虑
- 假定算法执行中总有 $\text{diam}(\square_w) > \epsilon/2 \cdot r^*$
- 若 $r \leq (1 + \epsilon)r^*$ ，则 $\text{diam}(\square_w) > \epsilon/2 \cdot r^* \geq \epsilon/(2(1 + \epsilon))r$
- 否则 $r > (1 + \epsilon)r^*$ 根据性质1， q^* 所在方格未被剪枝

(*) 对每个 $\square_w \in S$ ，若 $\|q - \text{rep}_w\| < r$ ，更新 $r = \|q - \text{rep}_w\|$ 及 $\hat{q} = \text{rep}_w$



有 $r \leq r^* + \text{diam}(\square_w)$ ，推出 $\text{diam}(\square_w) \geq r - r^* > r - r/(1 + \epsilon) = \epsilon/(1 + \epsilon) \cdot r$

* 其他可能的剪枝策略?

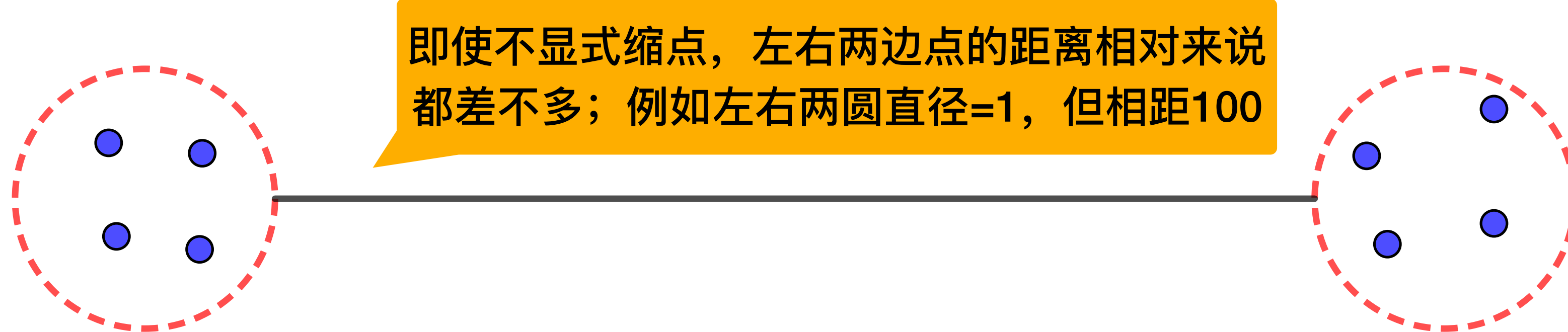
- 有同学提出：改成用 $r \leq (1 + \epsilon) \cdot \text{dist}(q, \square_i)$ 来断定 $\square_i$ 应该剪掉
- 与我们的剪枝条件 $r \leq (1 + \epsilon)(\|q - \text{rep}_i\| - \text{diam}(\square_i))$ 如何比较?
 - 是否有一者推出另一者?
 - 思考：求 $\text{dist}(q, \square_i)$ 需要多少时间？正确性？总运行时间？

3. 点对的离散化：WSPD

WSPD

直观讨论

- 输入点集 $P \subseteq [\Delta]^d$ ，设 $|P| = n$
- 考虑所有点对的距离（共有 $O(n^2)$ 对），试图将 所有点对之间的距离 离散化
- 直觉：设有 $S, T \subseteq P$ 两组点，且他们距离很远
 - 那么可以把 S 和 T 分别“缩点”，从而将 $S \times T$ 上的距离等于同一个值



WSPD

定义

- 一个 ϵ -WSPD是一系列 $\{\{A_i, B_i\}\}_i$, 使得

点集之间的距离等于最近点的距离, 即

$$\text{dist}(S, T) := \min_{x \in S, y \in T} \|x - y\|$$

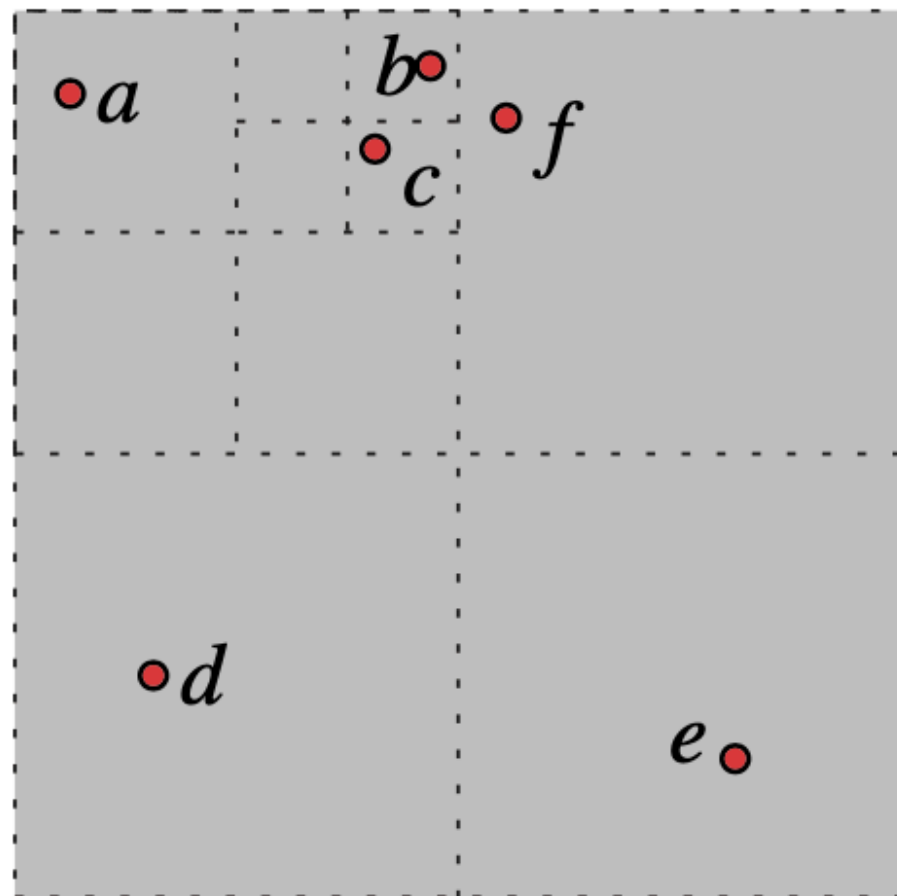
- $\forall i, A_i, B_i \subseteq P$ 且 $\max(\text{diam}(A_i), \text{diam}(B_i)) \leq \epsilon \cdot \text{dist}(A_i, B_i)$

即 A_i, B_i 相对直径来说
距离很远

- $\bigcup_i A_i \times B_i = P \times P$, 即任何 $p \neq q \in P$ 可以找到某个 i 使得 $p \in A_i, q \in B_i$

可以理解为对 $O(n^2)$ 个距离对的划分





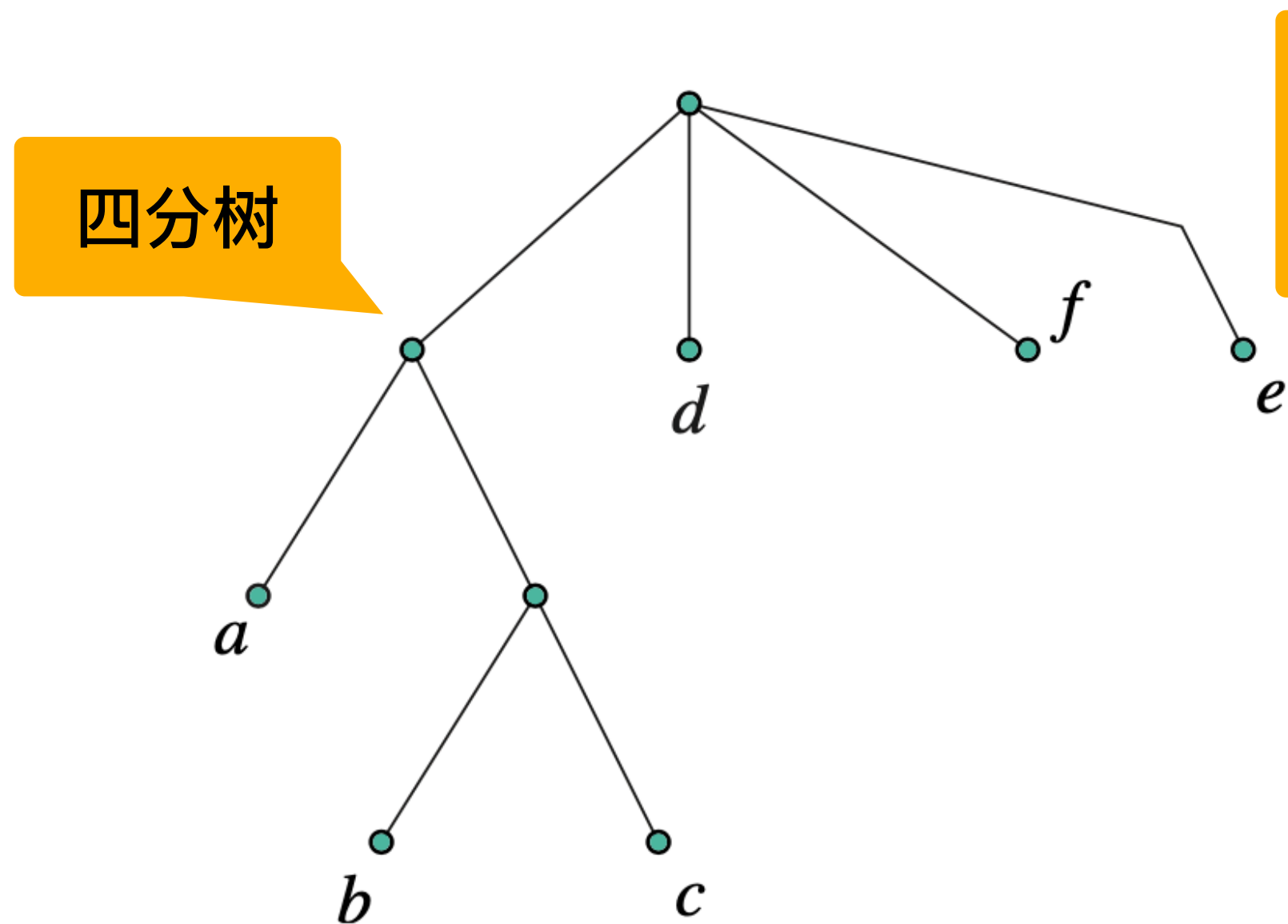
(i)

- $A_1 = \{d\}, B_1 = \{e\}$
 $A_2 = \{a, b, c\}, B_2 = \{e\}$
 $A_3 = \{a, b, c\}, B_3 = \{d\}$
 $A_4 = \{a\}, B_4 = \{b, c\}$
 $A_5 = \{b\}, B_5 = \{c\}$
 $A_6 = \{a\}, B_6 = \{f\}$
 $A_7 = \{b\}, B_7 = \{f\}$
 $A_8 = \{c\}, B_8 = \{f\}$
 $A_9 = \{d\}, B_9 = \{f\}$
 $A_{10} = \{e\}, B_{10} = \{f\}$

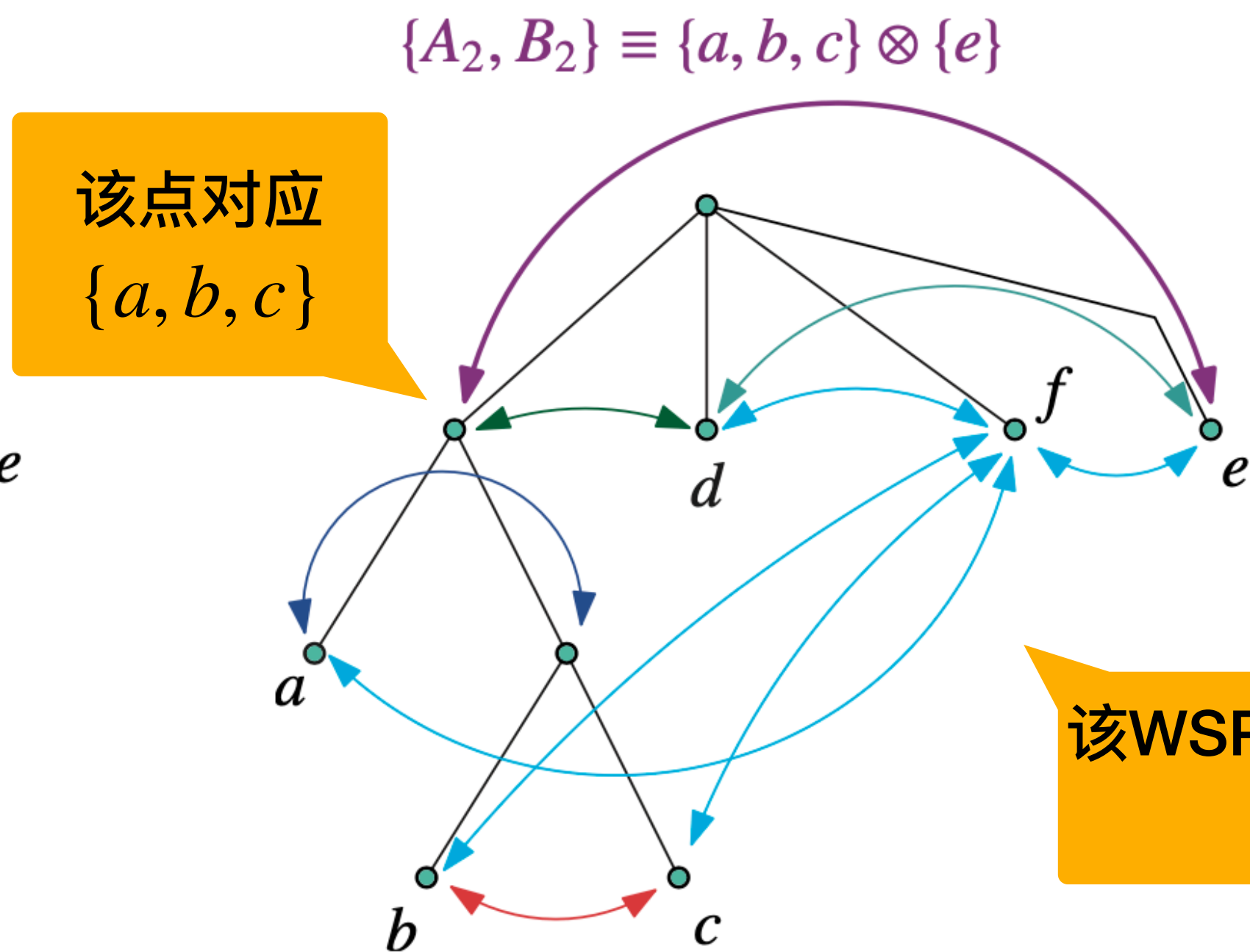
(ii)

$$\mathcal{W} = \left\{ \begin{array}{l} \{A_1, B_1\}, \\ \{A_2, B_2\}, \\ \{A_3, B_3\}, \\ \{A_4, B_4\}, \\ \{A_5, B_5\}, \\ \{A_6, B_6\}, \\ \{A_7, B_7\}, \\ \{A_8, B_8\}, \\ \{A_9, B_9\}, \\ \{A_{10}, B_{10}\} \end{array} \right\}$$

(iii)



(iv)



(v)

该WSPD可用四分树高效表示

基于四分树的WSPD：完整算法

WSPD用四分树的 $\square$ 表示，总复杂度 $O(n\epsilon^{-d} \log \Delta)$ ，所得WSPD有 $O(n\epsilon^{-d} \log \Delta)$ 项

全局对 $\square = [\Delta]^d$ 调用WSPD($\square, \square$)

WSPD($\square_u, \square_v$)

定义： $\delta(\square_u) = \text{diam}(\square_u)$ 若 $|\square_u \cap P| \geq 2$ ，否则 $\delta(\square_u) = 0$

if ($u = v$ and $\delta(\square_u) = 0$) return // 单点 $\square$ 不要配对到自己身上

if ($\delta(\square_u) < \delta(\square_v)$) swap u, v

if ($\delta(\square_u) \leq \epsilon \cdot \text{dist}(\square_u, \square_v)$)

dist直接看 $\square$ 而不是 $P \cap \square$ ；如何高效求？

return $\{\{\square_u, \square_v\}\}$

return $\bigcup_{i=1}^{2^d} \text{WSPD}(\square_{u_i}, \square_v)$

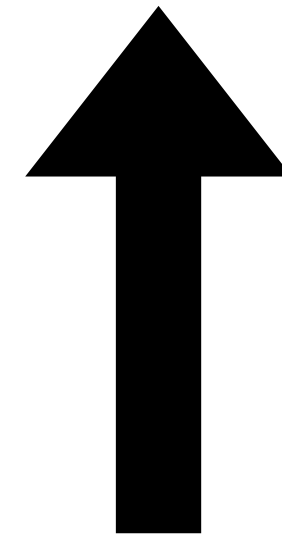
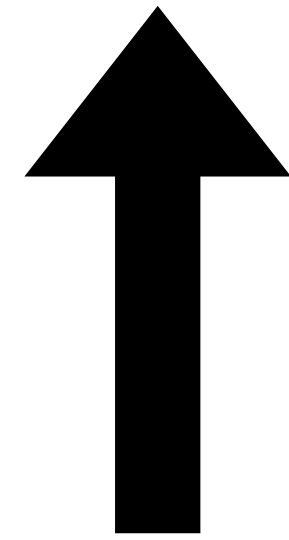
运行至此 $\{\square_u, \square_v\}$ 确实满足WSPD条件（注意到 $\delta(\square_u) \geq \delta(\square_v)$ ）
又因为算法会尝试一直递归到底，因此确保最后输出的是合法WSPD

$\square_{u_i}$ 是 $\square_u$ 的子 $\square$

* 算法性能分析

运行时间也是这个量级

- 总结论：算法输出的WSPD有 $O(n\epsilon^{-d} \log \Delta)$ 个 $\{A_i, B_i\}$



$\delta(\square_u) \geq \delta(\square_v)$

- 对每个四分树方格 $\square_v$ 考虑所有加入WSPD的 $\{\square_u, \square_v\}$ ，将对应的 $\square_u$ 加入 S_v
- **Claim:** $|S_v| \leq O(\epsilon^{-O(d)})$
- 利用Claim: 该四分树共 $O(n \log \Delta)$ 个（非空） $\square$ ，WSPD共有 $O(n\epsilon^{-d} \log \Delta)$ 项

* 算法性质

初始 $\square = [\Delta]^d$

- **性质:** 除初始调用($\square, \square$)外, 任何($\square_u, \square_v$)递归调用必是从($\hat{p}(\square_u), \square_v$)和($\square_u, \hat{p}(\square_v)$)之一产生的

$\hat{p}(\square)$ 表示 $\square$ 的父格子

- 可以推出: 任何递归调用($\square_u, \square_v$)满足 $\square_u$ 和 $\square_v$ 差至多一层
- 原因: 算法可以产生新递归调用的地方仅有两处

• if ($\delta(\square_u) < \delta(\square_v)$)

swap u, v

不改变 $\square_u, \square_v$, 只是交换

• return $\bigcup_{i=1}^{2^d} \text{WSPD}(\square_{u_i}, \square_v)$

下次调用中仅 $\square_u$ 被 $\square_u$ 的某个子 $\square$ 替代, 但 $\square_v$ 不改变

* 验证Claim

Claim: 对任何方格 $\square_v$, $|S_v| \leq O(\epsilon^{-O(d)})$

对四分树方格 $\square_v$, S_v 为算法中所有加入WSPD的 $\square_u$ 的集合

• 辅助结论: $\forall \square_u \in S_v$, $\text{dist}(\square_u, \square_v) \leq O(1/\epsilon) \cdot \delta(\square_u)$

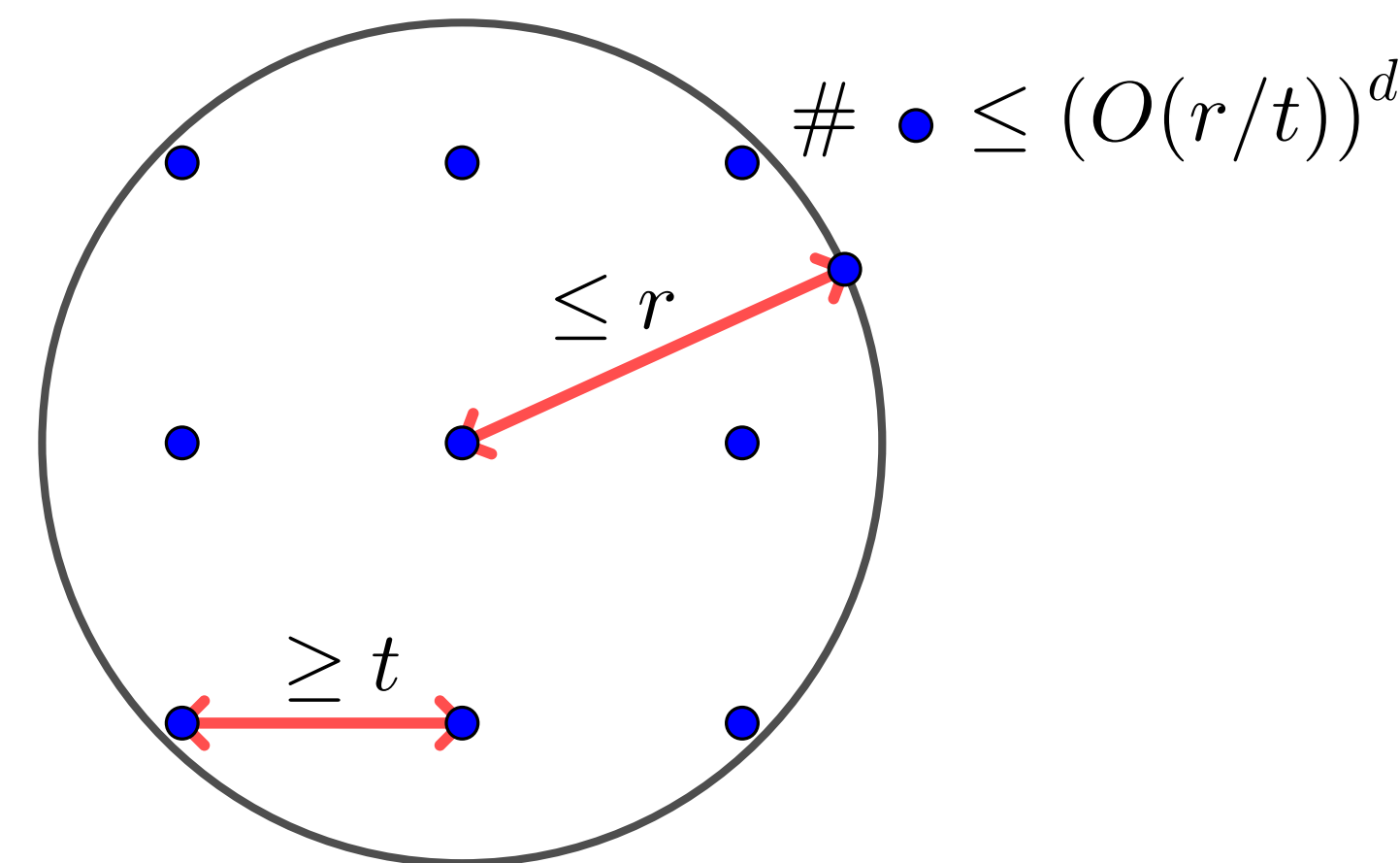
下页验证, 现在假定成立

• 若 $\delta(\square_u)$ 固定 = t , 则满足 $\text{dist}(\square_u, \square_v) \leq O(1/\epsilon) \cdot \delta(\square_u)$ 的 $\square_u$ 有几个?

• 任何两个这样 $\square_u$ 的格点中心距离至少 $\Omega(t)$

• 所有这些格点中心都在距离 $\square_v$ 的 $O(1/\epsilon) \cdot t$ 范围内

• 因此至多有 $\epsilon^{-O(d)}$ 个! 利用欧氏空间基本性质



• $\delta(\square_u) = t$ 的值有至多2种可能性, 因此 $|S_v| \leq O(\epsilon^{-O(d)})$

算法性质: $\square_u$ 和 $\square_v$ 至多差一层

* 验证辅助结论

$$\forall \square_u \in S_v, \text{dist}(\square_u, \square_v) \leq O(1/\epsilon) \cdot \delta(\square_u)$$

- 回忆算法步骤: $\text{if } (\delta(\square_u) \leq \epsilon \cdot \text{dist}(\square_u, \square_v)) \text{ return } \{\{\square_u, \square_v\}\}$
- 考虑一个递归调用 $(\square_u, \square_v)$ 令上述if成立 (即此 $\square_u \in S_v$)
- 此次 $(\square_u, \square_v)$ 递归调用一定就是从 $(\hat{p}(\square_u), \square_v)$ 和 $(\square_u, \hat{p}(\square_v))$ 之一产生的
- 且当次调用上述if判定为false

利用算法性质

否则就会return且不可能产生 $(\square_u, \square_v)$ 调用

* 验证辅助结论 cont.

$$\forall \square_u \in S_v, \text{dist}(\square_u, \square_v) \leq O(1/\epsilon) \cdot \delta(\square_u)$$

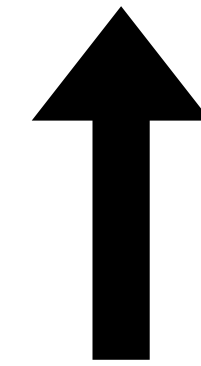
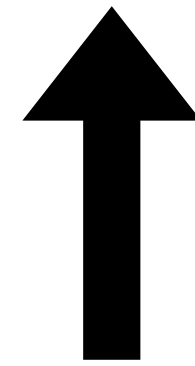
另一种情况($\hat{p}(\square_v), \square_u$)类似

if ($\delta(\square_u) \leq \epsilon \cdot \text{dist}(\square_u, \square_v)$)

- 设($\square_u, \square_v$)是从($\hat{p}(\square_u), \square_v$)调用产生的, 且“if”判断为false, 也就是

$$\delta(\hat{p}(\square_u)) > \epsilon \cdot \text{dist}(\hat{p}(\square_u), \square_v)$$

- 可以得出: $\text{dist}(\square_u, \square_v) \leq O(1/\epsilon) \cdot \delta(\hat{p}(\square_u))$ 进一步 $\leq O(1/\epsilon) \cdot \delta(\square_u)$



- 三角形不等式推出 $\text{dist}(\square_u, \square_v) \leq \text{dist}(\hat{p}(\square_u), \square_v) + \delta(\hat{p}(\square_u))$

- 代入 $\text{dist}(\hat{p}(\square_u), \square_v) < O(1/\epsilon) \cdot \delta(\hat{p}(\square_u))$ 后得出结论

补充：边界情况

- 刚刚的分析事实上需要假设 $\delta(\square_v) \neq 0$ ，也就是 $|\square_v \cap P| \geq 2$
- 如果 $|\square_v \cap P| = 1$ ，则不一定有 $\square_u$ 和 $\square_v$ 差一层
- 但依然不改变主要结果：对于同样大小的 $\square_u$ ，至多会有 $\epsilon^{-O(d)}$ 个放入WSPD
- 也就是每个 $\square_v$ 会放入 $\epsilon^{-O(d)} \log \Delta$ 个元素到WSPD
- 一共有 $O(n)$ 个这样的单点 $\square_v$
- 这部分总代价仍是 $n\epsilon^{-O(d)} \log \Delta$

WSPD应用：精确求最近点对

- 这是一个 $O(n \cdot 2^d \log \Delta)$ 时间的精确算法：
 - 构造 $\epsilon < 1$ 的WSPD $\{\{A_i, B_i\}\}_i$ $O(n \cdot 2^d \log \Delta)$ 时间
 - 对所有的 $\{A_i, B_i\}$ ，若 $|A_i| = |B_i| = 1$ ，记录 A_i 和 B_i 中唯一点的距离
 - 在所有距离中的最近点对就是答案 $O(n \cdot 2^d \log \Delta)$ 时间
- 原因：设最近点对是 (p, q) 且 $p \in A_i$ ， $q \in B_i$ ，则 $|A_i| = |B_i| = 1$

因此算法不会
遗漏最近点对

若有多余点 x 在 A_i ，则容易看出 $\|p - x\| < \|p - q\|$ ，矛盾



作业十三：最近点对精确求解

- <http://cssyb.openjudge.cn/26hw13/>
- 分值：2分
- Deadline: 5月13日

WSPD也可以用来近似直径

- 算法:

- 构造 $\epsilon/2$ -WSPD

- 对每个 $\{A_i, B_i\}$ 对, 分别找 rep_i^A , rep_i^B , 然后求 $\|\text{rep}_i^A - \text{rep}_i^B\|$

- 求所有 $\|\text{rep}_i^A - \text{rep}_i^B\|$ 里面最大的就是直径的 $(1 - \epsilon)$ -近似

- 设 q, s 两点是直径的两个端点 ($\|q - s\| = \text{diam}$), 且设 $q \in A_i$, $s \in B_i$

WSPD性质

$$\|\text{rep}_i^A - \text{rep}_i^B\| \geq \text{dist}(A_i, B_i) \geq \|q - s\| - \text{diam}(A_i) - \text{diam}(B_i) \geq (1 - \epsilon) \cdot \|q - s\|$$

根据定义

三角形不等式; 类似最近邻剪枝条件的场景

WSPD应用：求 $(1 + \epsilon)$ -Spanner

$$t \geq 1$$

- t -Spanner是数据集 $P \subseteq [\Delta]^d$ 的一个欧氏图的子图 H
 - 欧氏图：点集还是 P ，边集是 $P \times P$ ，边权重是端点间的欧氏距离
 - 子图：点集相同，边集是子集
 - t -Stretch：要求对任何 $x, y \in P$ 有 $\|x - y\| \leq \text{dist}_H(x, y) \leq t \cdot \|x - y\|$
 - 追求：给定 t ， H 满足 t -stretch条件，且边数尽量少
- 可看作是欧氏图的稀疏“压缩”，后续可使用稀疏图上的算法来解决欧氏问题

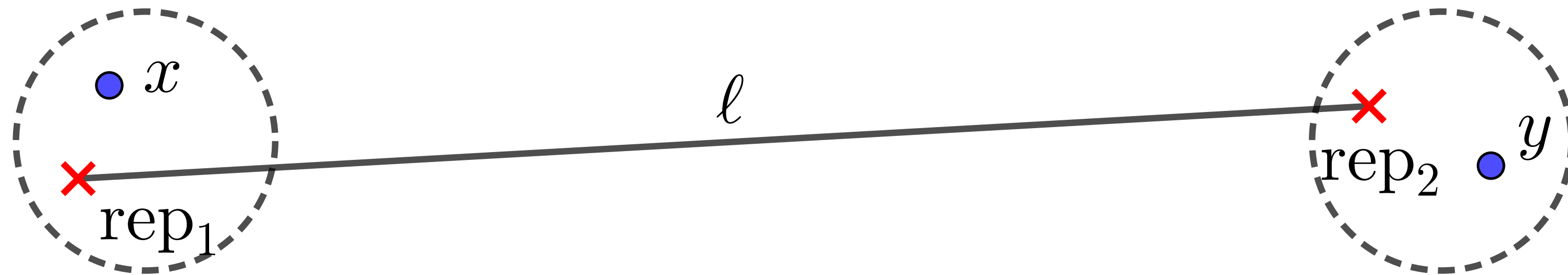
dist_H 代表 H 上的最短路距离

Spanner算法

- 算法：先求 $\epsilon/8$ -WSPD，再对每个 A_i, B_i 都找代表点 rep_1 和 rep_2 连边
- 因此总共是 $O(n\epsilon^{-d} \log \Delta)$ 条边

Spanner算法的分析

归纳法：按照 x, y 距离 D 递增进行归纳；假设是若 $\|x - y\| \leq D$ 则 x, y 满足 $(1 + \epsilon)$ -stretch



- 考虑一对点 x 和 y ，设 $x \in A_i$ ， $y \in B_i$ ，且 rep_1 和 rep_2 是代表点、其连边被加入H

不失一般性

- 观察： $\text{dist}(x, y) \geq \text{dist}(A_i, B_i) \geq 8/\epsilon \cdot \text{diam}(A_i) \geq 8/\epsilon \cdot \text{diam}(B_i)$

x, y 足够远

$$\ell = \text{dist}(\text{rep}_1, \text{rep}_2)$$

- 推出： $\ell \leq \text{dist}(A_i, B_i) + \text{diam}(A_i) + \text{diam}(B_i) \leq (1 + \epsilon/4) \cdot \text{dist}(x, y)$

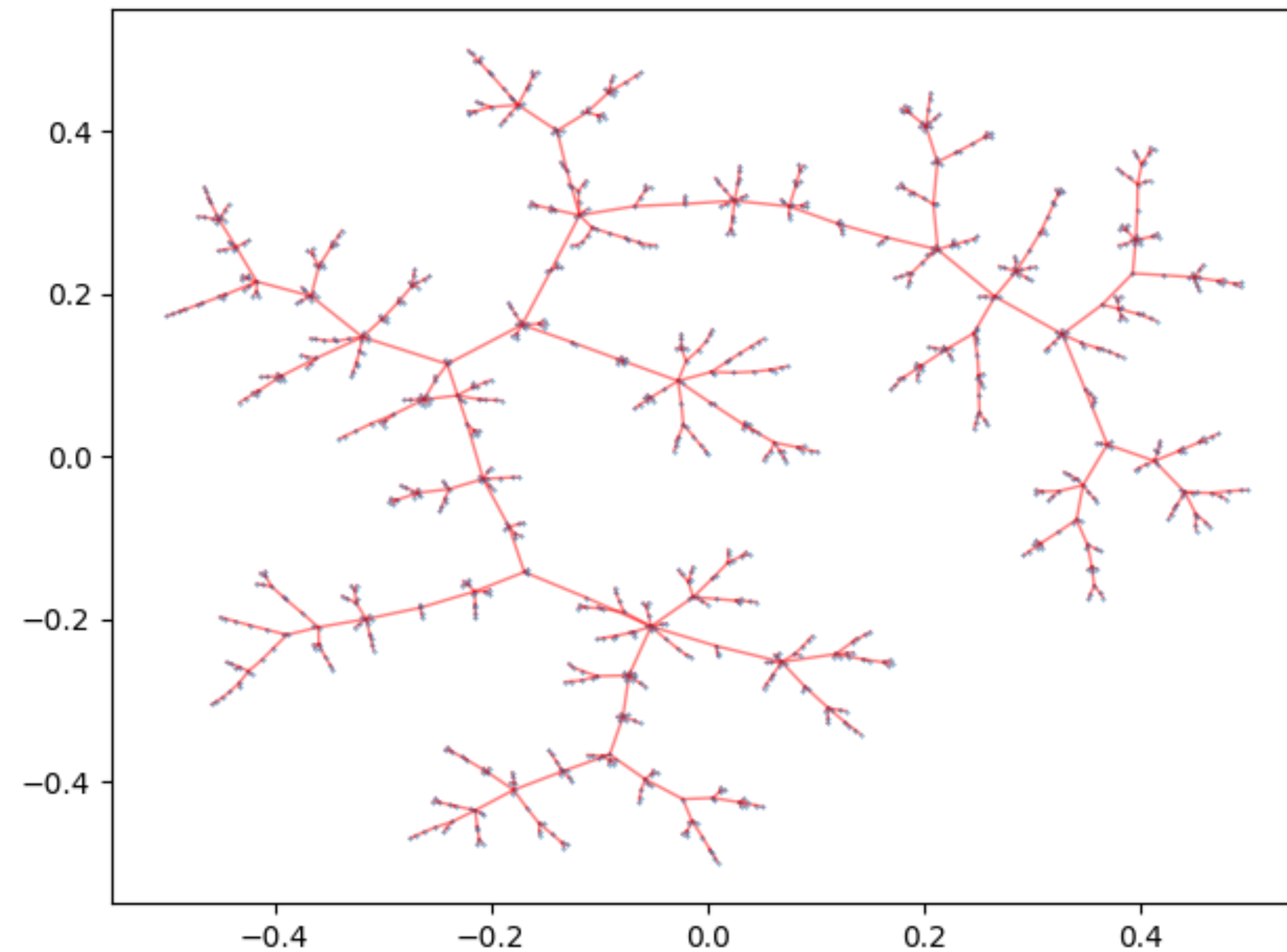
rep距离足够近似 x, y 距离

- 再利用归纳假设，
$$\begin{aligned} \text{dist}_H(x, y) &\leq \text{dist}_H(x, \text{rep}_1) + \text{dist}_H(y, \text{rep}_2) + \ell \\ &\leq 2(1 + \epsilon) \cdot \epsilon/8 \cdot \text{dist}(x, y) + \ell \leq (1 + \epsilon) \cdot \text{dist}(x, y) \end{aligned}$$

$\text{dist}_H(x, \text{rep}_1) \in (1 + \epsilon)\text{dist}(x, \text{rep}_1)$, rep_2 类似

欧氏最小生成树

- 图的最小生成树（MST）：最小权的使所有点连通的子图
- 欧氏数据点集 P 的最小生成树：定义在 P 的欧氏图上的MST



利用 $(1 + \epsilon)$ -Spanner求 $(1 + \epsilon)$ -近似最小生成树

- 算法：
 - 构造 P 的 $(1 + \epsilon)$ -spanner H
 - 在 H 上求MST，则该MST是一个 P 上的 $(1 + \epsilon)$ -近似的MST
- 解释：
 - 任何 P 上的MST可以对应于 H 上一个 $(1 + \epsilon)$ 倍权重的子连通图 H'
 - 因此 H 上的MST比 H' 要小，而 H 上的MST也是 P 上的生成树合法解

$$\text{MST}(H) \leq w(H') \leq (1 + \epsilon) \cdot \text{MST}(P)$$

H 是 $P \times P$ 的子图，因此 H 中的边都在 P 的欧氏图中

作业十四： $(1 + \epsilon)$ -近似欧氏最小生成树

- <http://cssyb.openjudge.cn/26hw14/>
- Deadline: 5月13日
- 2分